Cluster Computing (2026) 29:713
https://doi.org/10.1007/s10586-026-06514-x

Anomaly detection of automatic online extreme studentized deviate tests for pattern-changing streaming data

Yonghwan Kim[1] · **Sungwoo Kim**[1] · **Kichun Lee**[1]

Received: 4 July 2025 / Revised: 16 June 2026 / Accepted: 7 August 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

The package osESD in Python and R implements *online sequential extreme studentized deviate tests*, or osESD in short, a statistical testing method for anomaly detection in online streaming datasets with varying patterns. Its foundation lies in the general extreme studentized deviate test, or GESD test. The GESD test, a simple and effective anomaly detection test, lacks online learning and shows poor performance for pattern-varying data. Our proposed osESD algorithm uses sequential ESD, a variation of GESD, which implements online learning with change-rate trend vectors and residual trend vectors to deal with the problem of pattern-varying streaming data. We also automated parameter tuning in the package with multiple options the user can select for immediate deployment and use. We show that the model learning of our implementation is fast and the performance is excellent compared with numerous models with both various synthetic datasets and real datasets. Codes can be found in the PyPI [11].

Keywords Anomaly detection · Online algorithm · Streaming data · GESD

1 Introduction

Anomaly detection [3] is becoming an increasingly important task in industrial time series analysis. The extreme studentized deviate (**ESD**) test or Grubb's test [5] is a simple and efficient anomaly detection method for univariate datasets. The ESD test identifies anomalies or outliers by checking observations according to deviation from the center distribution. Those farthest away from the center are classified as anomalies. Simple and intuitive, the ESD test requires the user to specify the exact number of outliers, which is unrealistic in real-life data. To ensure clarity and consistency, we will treat 'anomaly' and 'outlier' as synonymous throughout this paper.

To overcome the limitations of ESD, a general ESD (GESD) [16] test has been used widely. The GESD test performs a hypothesis testing with a null hypothesis that no outliers exist and an alternative that more than a certain number n outliers exist. Like ESD, using deviation from the center, it targets the furthest points. For a time-series dataset with a total of $\mathcal{N}$ datapoints, it divides the dataset into sliding windows of equal size. In each of these windows, the index of data points is denoted as i. Then, the GESD package calculates test statistic R_i and cut-off value λ_i at point i, classifying it as an anomaly if $R_i > \lambda_i$, in which case, the point is removed from the dataset and calculations proceed until either $R_i < \lambda_i$ or the maximum number of r is reached. In other words, it performs a maximum of r separate tests.

While simple and effective, the GESD test also has its drawbacks. Like ESD, it relies heavily on data observations following a normal distribution and a proper setting of possible outliers. The original version is not applicable to pattern-varying time series, and neither an online learning mechanism nor a fast and effective offline training mechanism has been proposed.

These drawbacks prove detrimental in the real-life streaming time-series observations that are prevalent in modern industrial domains. Specifically, application servers deployed on a real-time basis demand that anomalies be detected swiftly and accurately. Typical offline learning procedures have limitations with such fast-growing

✉ Kichun Lee
skylee@hanyang.ac.kr

Yonghwan Kim
ghks5345@hanyang.ac.kr

Sungwoo Kim
iguv7447@hanyang.ac.kr

[1] Department of Industrial Engineering, Hanyang University, Seongdong-gu, Seoul, South Korea

Published online: 01 September 2026

datasets [18]. In addition, such datasets commonly have variable long and short-term trends and patterns, weakening the normality assumption that is essential in GESD tests.

Several attempts to improve anomaly detection have been made in the data-mining and machine-learning society using ESD tests. [23] presented a weighting clustering method to address the limitations of ESD tests, but it yielded poor results when the variance of dataset was either too high or too low. [19] from Twitter developed a long-term anomaly detection system that is suitable for cloud services. Their method uses GESD tests, seasonal trend decomposition, and piecewise approximation of the latent long-term trend to detect anomalies in datasets with seasonal trends. [2] proposed a technique for transforming time-series data from a sequence-bound linear form to a sequence-unbound form, making it possible to detect anomalies in time series with trends. However, these methods are offline algorithms and cannot detect anomalies in streaming data.

Accordingly, a substantial amount of research has also been made in the field of online anomaly detection using both statistical tests and machine learning algorithms. [20] combined auto-regressive models with wavelet analysis and proposed an online anomaly detection method called ARIMA-wavelet-HMM. [12] used centroid-based online anomaly detection in the presence of random noise. [1] proposed a two-phase online anomaly detection framework based on support vector clustering, a machine learning algorithm, showing satisfactory performance in nonstationary datasets.

More recently, [14] proposed Online-IForest, an extension of the classical Isolation Forest to streaming data that incrementally updates tree ensembles to adapt to evolving data without periodic retraining. [24] introduced IDK-S, an incremental distributional kernel for streaming anomaly detection that updates kernel-based distribution estimates to track distribution shifts. [21] proposed OML-AD, an online machine learning method for anomaly detection in non-stationary time series that updates predictive models instance by instance and scores anomalies from prediction errors.

This paper introduces the implementation of online sequential extreme studentized deviate tests, or osESD, providing a fast and effective training mechanism focused on streaming data that is either labeled or unlabeled. Our osESD test, based on the GESD test, is capable of handling streaming data via its fast offline initialization and online learning mechanism. The detection of outliers in time series with various short- and mid-term trends using the proposed package is possible by the combination of change-rate trend vectors and residual trend vectors at the core. In addition, to make it suitable for an online streaming environment, our method implements a sequential version of GESD in which test statistics R_i for both change-rate trend and residual trend are computed in an incremental fashion, demonstrating fast computational speed without sacrificing performance.

We also release a function that allows automated parameter tuning of our proposed osESD package. Named auto-osESD, it automates parameter tuning by itself and returns both the best parameters and results of the subsequent osESD anomaly test. In the absence of user-specified parameters, the package employs a set of predefined parameters. When unlabeled data is inserted, auto-osESD automatically injects pseudo y values into the beginning part of the data. With these and other unlabeled anomalies, the algorithm is able to find parameters appropriately and classifies the entire dataset. Specific explanations will be provided in section 4.3.

To the best of our knowledge and findings, an online GESD test targeting datasets with varying trends and equipped with a fast automated offline training mechanism that is provided in both a raw package and an automated parameter tuning package, has never been proposed in the related literature. Overall, our contribution including the extension and implementation of the GESD test, written in Python and R, (1) suitable for streaming time series with online learning, (2) can use change-rate trend vectors and residual trend vectors, and (3) allows for fast and effective training procedures to deal with varying patterns in the dataset.

We will make publicly available the implemented osESD package offered in the library osESD in Python and in the package osESD in R along with easy-to-run examples and datasets. Its core module (or function) name is osESD-Detector in both Python and R. Explanations about the code and other details are given in section 3.6.

The rest of the paper is organized as follows. Section 3 provides a brief explanation of GESD tests and our proposed method. Section 3.6 consists of various practical code examples using both languages Python and R. Section 4 presents the experiment settings and test results. Section 5 provides our conclusions and plans for future research. Finally, appendix with more detailed explanations are at 1.

2 Preliminary

We briefly introduce a generalized extreme studentized deviate (GESD) test in this section. The GESD test [16] is designed to find outliers in a univariate dataset assumed to follow a normal distribution. Its predecessor, the ESD test, has a huge drawback in that the number of suspected outliers k must be specified exactly and explicitly. Inaccurate k values could distort the results. On the other hand, the GESD test requires only an upper bound of the suspected anomalies, often denoted as r.

For the GESD test, let H_0 be the null hypothesis and H_a be the alternative hypothesis,

H_0: No outliers exist in the data set,

H_a: Up to *r* outliers exist in the data set.

Let S_1 be the initial set of observations $\{x_1, \ldots, x_n\}$ and *m* start with 1. The test statistic R_m for the *m*-th iteration ($m = 1, \ldots, r$) is calculated over the current active region S_m as follows:

$$R_m = \max_{x_j \in S_m} \frac{|x_j - \bar{x}_m|}{s_m}, \tag{1}$$

where $\bar{x}_m$ and s_m are the sample mean and sample standard deviation of the remaining dataset S_m, respectively. In each *m*-th iteration, the test identifies the observation x_j that maximizes $|x_j - \bar{x}_m|$ and removes it to form the next subset S_{m+1}. It then recalculates the test statistic with the remaining $n - m$ data points, updating *m* to $m + 1$. This process repeats until *r* points are removed, yielding R_1, R_2,..., R_r. Next, it calculates the critical cut-off value λ_m, as follows:

$$\lambda_m = \frac{(n-m) \times t_{p,n-m-1}}{\sqrt{(n-m-1+t^2_{p,n-m-1}) \times (n-m+1)}}, \tag{2}$$

where $m = 1, 2, ..., r$ and $t_{p,v}$ is the 100*p* percentage point from the *t* distribution with *v* degrees of freedom and $p = 1 - \frac{\alpha}{2(n-m+1)}$. Simulation studies indicate that this critical λ_m value approximation is very accurate when $n \geq 25$ and reasonably accurate when $n \geq 15$. The number of outliers is determined by finding the largest *m* that satisfies $R_m > \lambda_m$ [16].

Simple and effective, the GESD test still has certain limitations. It cannot fit streaming time-series data due to the nature of the original model. Thus, it cannot effectively find anomalies in datasets with locally varying trends. This limitation is inherent because the test is based on a test statistic that relies on the sample mean and the sample standard deviation being under the normal assumption.

3 Proposed automated sequential ESD test

In this section, the GESD test is extended to a sequential version with offline training and online updating and finally with automated parameter tuning.

3.1 Problem formulation

Before detailing the proposed model, we explicitly present the mathematical formulation of the hypothesis test. The anomaly detection framework sequentially processes data in sliding windows.

Instead of testing whether an individual newly arrived data point is an outlier in isolation, our method performs the hypothesis test at a window level. Let $\mathcal{W}$ represent a given sliding window of time-series data. The null hypothesis H_0 and the alternative hypothesis H_a for $\mathcal{W}$ are defined as follows:

- H_0: No outliers exist within the current window $\mathcal{W}$.
- H_a: Up to *r* outliers exist within the current window $\mathcal{W}$.

If the hypothesis test rejects H_0 and identifies a set of outliers within $\mathcal{W}$, a specific data point is classified as an anomaly if it belongs to this identified set. This window-level formulation allows us to capture varying patterns before making point-level anomaly decisions. Although we base our procedure on ideas from hypothesis testing, the primary goal of our method is to solve a binary classification problem that decides whether each observation is normal or anomalous. Accordingly, we use hypothesis testing as a modeling tool to design window-level decision rules, while evaluating the overall performance with standard classification metrics.

3.2 Sequential ESD test for varying patterns

We propose a new model utilizing the GESD test that solves the two problems mentioned above. Our model, online sequential extreme studentized deviate test, or osESD for short, uses an online variation of the GESD test for online learning and newly added transform vectors to deal with varying patterns.

The original GESD test calculates all R_1, R_2,..., R_r values before comparing them to λ_m to find the number of anomalies *m*, being the max *m* that satisfies $R_m > \lambda_m$. However, our osESD does not calculate all *r* of R_m values to find this max *m*. Instead, we initially set *m* to 1 to obtain R_1 and compare it immediately with λ_1. If $R_m > \lambda_m$, we update *m* to $m + 1$ and repeat the process. Otherwise, we stop the test and conclude $m - 1$ as the number of anomalies in the data. The iteration of this process is no more than the upper bound *r*. With this change in the process, whenever a new data point is presented in the streaming data, we need not perform all *r* individual tests in the sequential ESD test, dramatically reducing the computation time.

In the first step, we assume the initial time-series data is sufficiently large. We denote this dataset as $\mathcal{X} = (x_1, x_2, ..., x_N)$ with length $\boldsymbol{N}$. Then for $i \in \{1, \cdots, N - k + 1\}$, we split the fully prepared offline dataset into a sliding window $\mathcal{X}_{i|k} = (x_i, x_{i+1}, ..., x_{i+k-1})$ and its time stamps $\mathrm{t}_{i|k} = (t_i, t_{i+1}, ..., t_{i+k-1})$ for a window size *k*, and these are transformed into a change-rate trend

vector and residual trend vector. The change-rate vector targets short-term patterns and the residual trend targets mid-term patterns. These components are incorporated into our method to facilitate the identification of anomalies within data exhibiting diverse patterns or trends, which is usually challenging to discern using conventional parametric models. Using $\mathcal{X}_{i|k_1}$ for window length k_1, we denote the initial change-rate trend vector as $\mathrm{c}_1 = (c_1, ..., c_{N-k_1+1})$ with

$$c_i = \frac{x_{i+k_1-1} - x_i}{t_{i+k_1-1} - t_i}, \qquad i = 1, 2, ..., N - k_1 + 1, \tag{3}$$

where x_i is the observed value at time i. This change-rate vector c_i for $\mathcal{X}_{i|k_1}$ represents the average change between k_1 leaps in $\mathcal{X}$, with the difference of time in the denominator and difference of observation values in the numerator.

In addition to this change-rate vector, using $\mathcal{X}_{i|k_2}$ for window length k_2, we utilize the initial residual trend vector $\mathrm{r}_1 = (r_1, ..., r_{N-k_2+1})$ where $r_i = x_{i+k_2-1} - \hat{x}_{i+k_2-1}$ for $i \in \{1, \cdots, N - k_2 + 1\}$ as well and $\hat{x}_{i+k_2-1}$ is the predicted value at time point t_{i+k_2-1} using regression within each window $\mathcal{X}_{i|k_2}$. Specifically, for $\bar{x}_{i|k_2} = \sum_{j=1}^{k_2} x_{i+j-1}/k_2$, $\bar{t}_{i|k_2} = \sum_{j=1}^{k_2} t_{i+j-1}/k_2$, we obtain

$$\hat{x}_i = \bar{x}_i + (t_{i+k_2-1} - \bar{t}_{i|k_2}) \frac{\mathrm{cov}\ (\mathrm{t}_{i|k_2}, \mathcal{X}_{i|k_2})}{\mathrm{cov}\ (\mathrm{t}_{i|k_2}, \mathrm{t}_{i|k_2})}, \tag{4}$$

where $\mathrm{cov}\ (\mathrm{x}, \mathrm{y}) = \sum_j (x_j - \bar{x})(y_j - \bar{y})$ is a covariance between the two vectors, x and y. The residual trend vector describes the degree of change within k_2 leaps in $\mathcal{X}$, interpreted as a within-k_2 trend vector. Thus, the change-rate trend vector window size is k_1 and residual trend vector window size is k_2. We notice that hyperparameters k_1 and k_2 depend on the domain problem and we will propose an automated way to set them.

Then, we perform the sequential ESD (sESD) test internally with the above values according to the following procedures. For $m \geq 1$, we find the index $\mathrm{J}_{\mathrm{c}_1}^{(m)}$ being the maximum among the standardized values in c_1, setting the maximum as $\mathrm{R}_{\mathrm{c}_1}^{(m)}$,

$$J_{\mathrm{c}_1}^{(m)} = \arg\max_i \left\{ \frac{|c_i - \bar{c}_1|}{s_{\mathrm{c}_1}} \right\}, \ R_{\mathrm{c}_1}^{(m)} = \max_i \left\{ \frac{|c_i - \bar{c}_1|}{s_{\mathrm{c}_1}} \right\}, \tag{5}$$

where $\bar{c}_1$ and s_{c_1} are the sample average and the sample standard deviation of c_1, respectively. In other words, $\mathrm{R}_{\mathrm{c}_1}^{(m)}$ is the maximum R statistic value in the current transformed change-rate vector c_1 and its index is $J_{\mathrm{c}_1}^{(m)}$. If $\mathrm{R}_{\mathrm{c}_1}^{(m)}$ exceeds critical value λ_m, then we regard observation $x_{J_{\mathrm{c}_1}^{(m)}}$ at time $J_{\mathrm{c}_1}^{(m)}$ as an anomaly. We continue this process until either $R_m \leq \lambda_m$ or m reaches the upper bound r. We do the same with residual rate trend vector r_1 as above, similarly obtaining the index $J_{\mathrm{r}_1}^{(m)}$ and the maximum $R_{\mathrm{r}_1}^{(m)}$:

$$J_{\mathrm{r}_1}^{(m)} = \arg\max_i \left\{ \frac{|r_i - \bar{r}_1|}{s_{\mathrm{r}_1}} \right\}, \ R_{\mathrm{r}_1}^{(m)} = \max_i \left\{ \frac{|r_i - \bar{r}_1|}{s_{\mathrm{r}_1}} \right\}. \tag{6}$$

Given the two trend vectors, c_1 and r_1, a detection strategy is either a conservative detection by deciding an anomaly if detected in both the vectors or a generous one if detected in one of the two vectors, which brings a parameter 'choice condition' in the follow-up tuning stage. If deemed an anomaly, we replace it with the predicted value $\hat{x}_N$ at position t_N using linear regression.

As a new online observation x_{N+1} arrives, the algorithm computes the newest elements c_{N-k_1+2} and r_{N-k_2+2} . The trend vectors are updated incrementally by discarding the oldest elements, yielding $\mathrm{c}_2 = (c_2, \ldots, c_{N-k_1+2})$ and $\mathrm{r}_2 = (r_2, \ldots, r_{N-k_2+2})$. With these updated vectors, the sample statistics are recalculated in $\mathcal{O}(1)$ time, and the sESD test described above is repeated to determine whether x_{N+1} is an anomaly. This sequential detecting and replacing process continues as new data stream in, leading to faster execution time compared to the original GESD tests.

In summary, the operational process of our implementation is depicted in Fig. 1. The framework consists of two main steps: an offline initialization step and an online detection and learning step.

3.3 Statistical interpretation

We provide the statistical interpretation of the two trend vectors. We assume that the observations $t_i \leq T$ are normally distributed by a locally linear trend and the linear trend for $t_i > T$ changes as follows:

$$x_i \sim N(\beta_0 + \beta_1 t_i, \sigma^2), \ t_i \leq T,$$
$$x_i \sim N(\beta_0' + \beta_1' t_i, \sigma^2), \ t_i > T,$$

where $\beta_0 \neq \beta_0'$ and $\beta_1 \neq \beta_1'$. That is to say, $T + 1$ is a time point of distribution change.

For change-rate trend vector c, let us assume time stampes are equally spaced with $t_j = j$ for the sake of simplicity. Then, change rates follow normal distribution with $c_i \sim N(\beta_1, 2\sigma^2/(k_1 - 1)^2)$ if $i \leq T - k_1 + 1$, which means sliding window $\mathcal{X}_{i|k_1}$ is within times before the distribution change. Furthermore, the distribution of c_{T-k_1+2} shifts to $c_{T-k_1+2} \sim N([(\beta_0' - \beta_0) + \beta_1'(T + 1) - \beta_1(T - k_1 + 2)]/(k_1 - 1), 2\sigma^2/(k_1 - 1)^2)$.Thus, the GESD with the change-rate trend c finds a locally linear change, mainly slope change for sufficiently large k_1, in dataset $\mathcal{X}$.

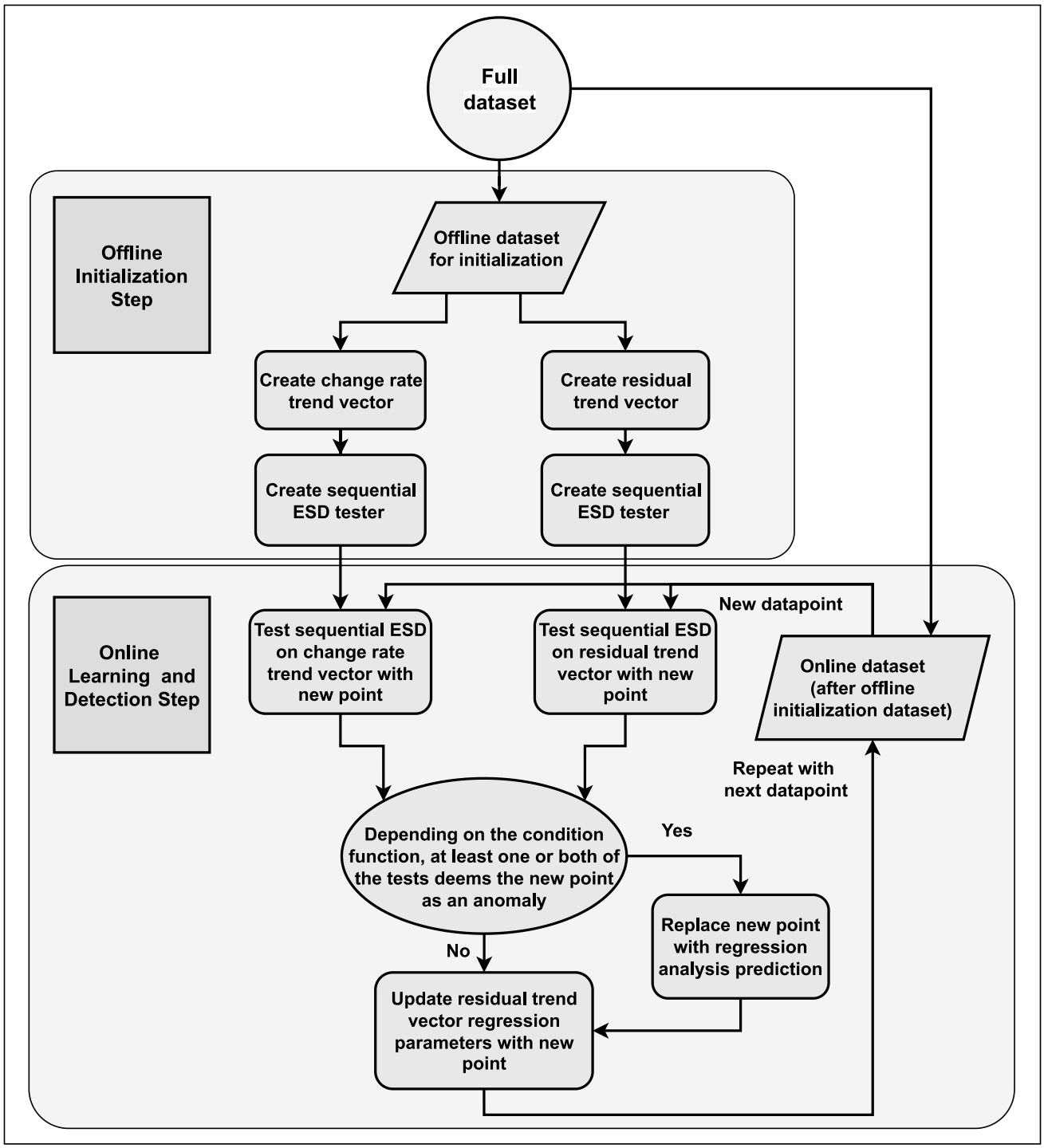


Fig. 1 Architecture of sequential ESD

For residual trend vector r, we prove that residual trends are normally distributed with expectations varying according to local linearity changes. We construct data input matrix $X_0 = [1_{k_2}\ \mathrm{t}_{T-k_2+1|k_2}]$ of size $k_2 \times 2$ for times $\mathrm{t}_{T-k_2+1|k_2}$, where 1_ℓ is an $\ell \times 1$ vector of which elements are all ones. Then, for $\hat{\boldsymbol{\beta}}_0 = (X_0^T X_0)^{-1} X_0^T \mathcal{X}_{T-k_2+1|k_2}$, the residual r_{T-k_2+1} becomes $r_{T-k_2+1} = x_T - [1\ t_T]\hat{\boldsymbol{\beta}}_0$. Using $E[\hat{\boldsymbol{\beta}}_0] = [\beta_0\ \beta_1]^T$ and $Cov[\hat{\boldsymbol{\beta}}_0] = (X_0^T X_0)^{-1}\sigma^2$, the distribution is

$$r_{T-k_2+1} \sim N\left(0, (1 - [1\ t_T](X_0^T X_0)^{-1}[1\ t_T]^T)\sigma^2\right). \tag{7}$$

Next, for residual r_{T-k_2+2} including x_{T+1} of which the distribution changed from that of x_T, we express $\mathcal{X}_{T-k_2+2|k_2}$

and $\mathrm{t}_{T-k_2+2|k_2}$ by $\mathcal{X}_{T-k_2+2|k_2} = [\mathcal{X}^T_{T-k_2+2|k_2-1}\ x_{T+1}]^T$ and $\mathrm{t}_{T-k_2+2|k} = [\mathrm{t}^T_{T-k_2+2|k-1}\ t_{T+1}]^T$, respectively. In a similar fashion, we express the data input matrix X_1 as

$$X_1 = \begin{bmatrix} X_{11} \\ X_{12} \end{bmatrix},$$
$$X_{11} = [1_{k_2-1}\ \ \mathrm{t}_{T-k_2+2|k_2-1}],\ X_{12} = [1\ t_{T+1}].$$

Notice that X_{11} is of size $(k_2-1)\times 2$ and X_{12} of 1×2 , which brings the size of X_1 to be $k_2\times 2$. Then, by applying the Woodbury formula to $(X_1^TX_1)^{-1}$ with X_{11} and X_{12} in $\hat{\boldsymbol{\beta}}_1 = (X_1^TX_1)^{-1}X_1^T\mathcal{X}_{T-k_2+2|k_2}$, we obtain the residual r_{T-k_2+2} as follows:

$$r_{T-k_2+2} = \frac{x_{T+1} - X_{12}^T\hat{\beta}'_{11}}{1 + X_{12}(X_{11}^TX_{11})^{-1}X_{12}^T},$$

where $\hat{\beta}'_{11} = (X_{11}^TX_{11})^{-1}X_{11}^T\mathcal{X}_{T-k_2+2|k_2-1}$. Thus, its distribution is

$$r_{T-k_2+2} \sim N\Big(\frac{(\beta_0-\beta'_0)+t_{T+1}(\beta_1-\beta'_1)}{1+X_{12}(X_{11}^TX_{11})^{-1}X_{12}^T}, \frac{\sigma^2}{(1+X_{12}(X_{11}^TX_{11})^{-1}X_{12}^T)}\Big). \tag{8}$$

Comparing the distribution of r_{T-k_2+1} in (7) and that of r_{T-k_2+2} in (8), we observe that the expectation changes by both the slope and the intercept at the local distribution change at T which will be It means that residual trend r_i captures the difference of both intercept and slope in the model, and the expectation changes are captured in the GESD test.

3.4 Computation

We also made quicker calculations possible by using an online updating algorithm. This made calculating the residual parameters take a time complexity of $\mathcal{O}(1)$ instead of the usual $\mathcal{O}(N)$. As explained in Section 3.2, we first transform the offline initialization data to a change-rate trend vector c and residual trend vector r, and then test it online using sequential ESD. During the transformation for the within-k_2 trend vector, we need to compute linear-regression parameters which would normally take a time complexity of $\mathcal{O}(N)$. However, since the residuals are calculated sequentially one by one and only one value changes from the previous vector that we calculated the parameters with, we use an online updating algorithm for faster calculation.

For that purpose, when feeding new x_{i+k_2} into $\mathcal{X}_{i|k_2}$ and making $\mathcal{X}_{i+1|k_2}$, we calculate new values from the previous values in a constant time complexity $\mathcal{O}(1)$ as follows:

$$\begin{aligned} \bar{t}_{i+1|k_2} &\leftarrow \bar{t}_{i|k_2} + \frac{t_{i+k_2}-t_i}{k_2}, \\ \bar{x}_{i+1|k_2} &\leftarrow \bar{x}_{i|k_2} + \frac{x_{i+k_2}-x_i}{k_2} \end{aligned} \tag{9}$$

$$\begin{aligned} \mathrm{cov}\ (\mathrm{t}_{i+1|k_2}, \mathrm{t}_{i+1|k_2}) \leftarrow\ & \mathrm{cov}\ (\mathrm{t}_{i|k_2}, \mathrm{t}_{i|k_2}) \\ &+ \frac{(t_{i+k_2}-\bar{t}_{i+1|k_2})^2 - (t_i-\bar{t}_{i|k_2})^2}{k_2}, \end{aligned} \tag{10}$$

$$\begin{aligned} \mathrm{cov}\ (\mathrm{t}_{i+1|k_2}, \mathcal{X}_{i+1|k_2}) \leftarrow\ & \mathrm{cov}\ (\mathrm{t}_{i|k_2}, \mathcal{X}_{i|k_2}) \\ &+ \frac{(t_{i+k_2}-\bar{t}_{i+1|k_2})(x_{i+k_2}-\bar{x}_{i+1|k_2}) - (t_i-\bar{t}_{i|k_2})(x_i-\bar{x}_{i|k_2})}{k_2}. \end{aligned} \tag{11}$$

The above calculations in addition to (4) accelerate the update of residual r_{i+1} from r_i in an online manner. Then, using updated residual trend vector r_2, for example $\mathrm{r}_2 = (r_2, \ldots, r_{N-k_2+2})$, we update the ESD statistics, $J^{(m)}_{\mathrm{r}_2}$ and $R^{(m)}_{\mathrm{r}_2}$ as in (6), with constant time complexity $\mathcal{O}(1)$.

The time complexity per step involves trend vector updates and sequential detection. Both trend vectors are updated in $\mathcal{O}(1)$ time. Specifically, the change-rate involves a simple slope calculation in (3), and residual parameters follow the update rules in (9)–(11). Following these updates, the search for the maximum R statistic within a window of size N requires $\mathcal{O}(N)$ operations. Since this search repeats up to r times, the formal per-step complexity is $\mathcal{O}(r\cdot N)$.

Because N and r are fixed parameters independent of the stream length T, the algorithm scales linearly with data volume, leading to an overall time complexity of $\mathcal{O}(T\cdot r\cdot N)$. Regarding memory, the algorithm only maintains the current sliding windows and associated vectors, which keeps the space complexity at $\mathcal{O}(N)$.

3.5 Automated parameter tuning

The performance of the proposed osESD implementation heavily relies on setting the parameters. The parameters that require tuning are shown in Table 1.

In fact, users often adopt a grid search that is a basic parameter-tuning strategy for their domain-specific problems and datasets. We relieve this burden by providing AUTO_OSESD, written in both PYTHON and R, that automates parameter-tuning. Using a fixed amount of label

Table 1 Parameters for osESD

Parameter	Meaning
Window size	Size of the window used in offline initialization and online learning steps
Choice condition	Choice function when determining anomalies with both vectors
maxr	Max number of anomalies in each window
dwin	Size of change-rate trend vector
rwin	Size of within-k_2 trend vector
Alpha	Alpha value of hypothesis test

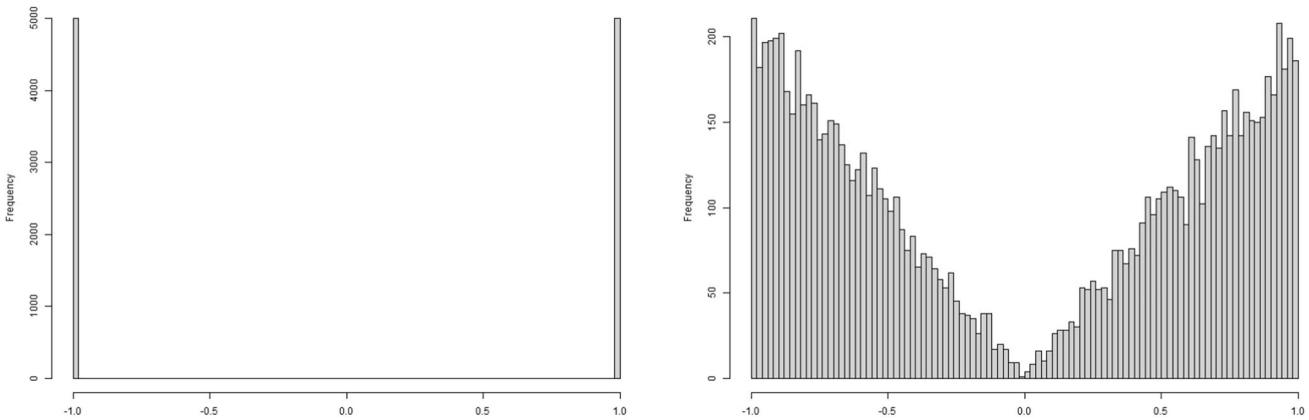
perturbation for parameter tuning, it finds the best parameters, then determining anomalies on the entire dataset based on these parameters.

In specific, when input datasets are unlabeled, AUTO_osESD automatically inserts generated labels, pseudo anomaly y, at the beginning part of the dataset, finding suitable parameters using both the original unlabeled and the generated labeled data. The number of adjusted labels is set to be 2 percent of the beginning part by default. It is implemented by adjusting the original data y_{p_i} to anomaly point y'_{p_i} for p_i being the ith index of anomaly generated, according to the following formula:

$$y'_{p_i} = y_{p_i} + M \times s_{p_i} \times z_{1-\alpha/2}, \tag{12}$$

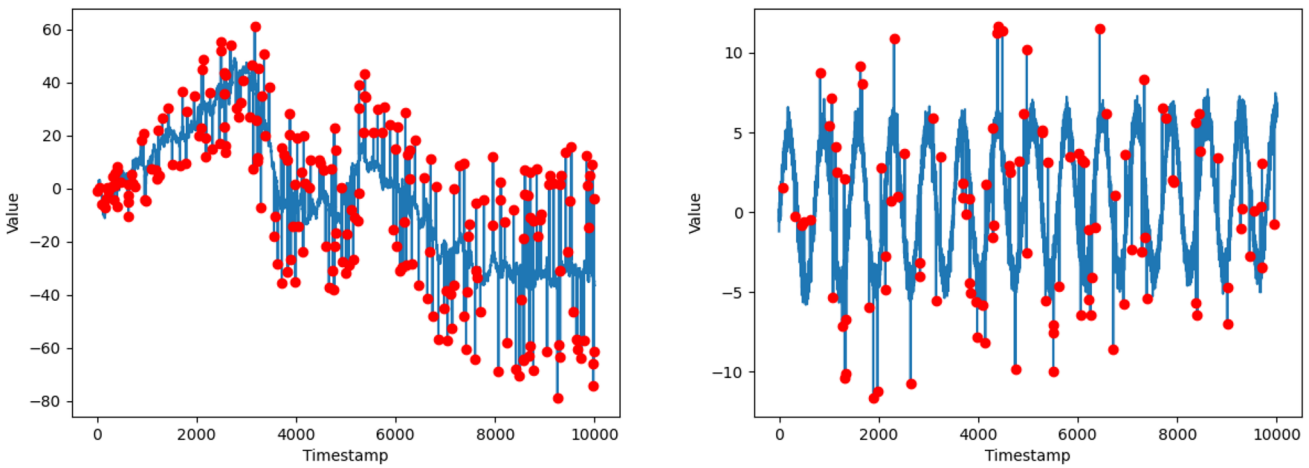
where M stands for a multiplier that is sampled from one of two probability distributions, Bernoulli distribution with domain $\{-1, 1\}$ and a distribution with a linearly increasing from zero to ± 1. Figure 2 shows their empirical probability distributions. We designed the sampled multiplier to be close to 1 or -1 and far from zero, such that M represents both the direction and the magnitude of a generated anomaly. In addition, s_{p_i} is the sample standard deviation of the data subset $y_{p_{i-1}}, y_{p_{i-1}+1}, ..., y_{p_i-1}$, and $z_{1-\alpha/2}$ is the $(1-\alpha/2)$-quantile of the standard normal distribution with α being 0.05 by default.

Using these labeled data parts, the method identifies the optimal parameter set by maximizing a weighted objective function that combines Precision, Recall, F1-score, and computational efficiency (measured as negative execution time). This allows the tuning process to prioritize specific metrics according to user-defined weights, ensuring that the selected parameters provide a robust balance between detection accuracy and the real-time constraints of the streaming environment.



(a) Bernoulli distribution (b) Linear distribution

Fig. 2 Sampling distributions for the multiplier in the anomaly generation



(a) ARIMA dataset (b) Seasonal dataset

Fig. 3 ARIMA dataset and seasonal dataset example

Table 2 Test results of comparison tests

Measure	osESD	IF	RRCF	AE	VAE	Z-Score	HW-A	LSTM-ED
(a) Total								
Recall	**0.791**	0.350	0.209	0.435	0.554	0.640	0.576	0.673
Precision	**0.795**	0.348	0.362	0.288	0.342	0.481	0.778	0.503
F1-score	**0.762**	0.347	0.224	0.252	0.344	0.509	0.513	0.502
Type I Error	**0.002**	0.008	0.008	0.040	0.030	**0.002**	0.012	0.015
Time (sec)	2.601	0.207	36.523	11.113	3.044	**0.001**	0.088	6.032
(b) A1Benchmark								
Recall	0.573	0.662	0.278	0.599	0.765	0.421	0.358	**0.819**
Precision	0.516	**0.656**	0.441	0.248	0.314	0.382	0.500	0.351
F1-score	0.475	**0.655**	0.275	0.262	0.403	0.321	0.328	0.440
Type I Error	0.008	0.007	0.007	0.011	0.005	0.008	0.017	**0.004**
Time (sec)	0.601	0.132	10.026	3.557	0.970	**0.001**	0.034	1.998
(c) A2Benchmark								
Recall	**0.920**	0.615	0.261	0.520	0.762	0.464	0.314	0.784
Precision	**0.901**	0.630	0.850	0.472	0.465	0.741	0.843	0.503
F1-score	**0.905**	0.617	0.362	0.364	0.498	0.522	0.378	0.526
Type I Error	**0.000**	0.001	0.007	0.040	0.019	0.003	0.014	0.020
Time (sec)	0.519	0.125	23.659	3.598	0.980	**0.001**	0.033	2.025
(d) A3Benchmark								
Recall	**0.600**	0.255	0.155	0.451	0.458	0.710	0.309	0.471
Precision	**0.661**	0.251	0.320	0.307	0.356	0.165	0.992	0.377
F1-score	**0.600**	0.251	0.192	0.275	0.315	0.252	0.451	0.325
Type I Error	0.004	0.004	0.008	0.036	0.022	**0.000**	0.016	0.027
Time (sec)	0.815	0.235	24.398	10.554	3.167	**0.001**	0.038	6.105
(e) A4Benchmark								
Recall	**0.589**	0.160	0.102	0.381	0.399	0.495	0.196	0.429
Precision	**0.858**	0.155	0.211	0.298	0.366	0.164	0.844	0.358
F1-score	**0.656**	0.157	0.127	0.232	0.256	0.227	0.296	0.260
Type I Error	0.006	0.004	0.009	0.039	0.032	**0.000**	0.024	0.027
Time (sec)	0.4121	0.150	11.317	4.093	1.145	**0.001**	0.037	2.314
(f) ARIMA dataset								
Recall	**0.992**	0.197	0.241	0.235	0.330	0.890	0.981	0.675
Precision	**0.951**	0.194	0.120	0.221	0.262	0.814	0.787	0.624
F1-score	**0.970**	0.196	0.160	0.168	0.229	0.848	0.868	0.626
Type I Error	**0.000**	0.016	0.008	0.040	0.034	0.002	**0.000**	0.008
Time (sec)	4.767	0.283	82.564	24.650	6.667	**0.001**	0.193	13.250
(g) Seasonal state-space								
Recall	0.999	0.314	0.244	0.474	0.677	0.959	**1.000**	0.912
Precision	**0.787**	0.305	0.247	0.163	0.274	0.822	0.617	0.755
F1-score	0.869	0.309	0.245	0.208	0.382	**0.882**	0.755	0.818
Type I Error	**0.000**	0.018	0.008	0.072	0.065	0.001	**0.000**	0.012
Time (sec)	8.230	0.370	72.665	24.454	6.725	**0.001**	0.188	13.161

3.6 Software implementation

We expound the software implementations of our proposed anomaly detection package osESD, for both Python and R. The implementations in Python and R yielded identical results, as expected, for the same scenarios. However, we observed that the runtimes of the R implementation were much shorter than that of Python. For Python, we uploaded our package osESD to PyPI, the link being [11]. The details are provided in Appendix 1. For R, we upload our package to a Github repository [10]. The details are provided in Appendix 2. Section 4 presents the details of experiments such as hardware settings, datasets used, comparison tests, and plotted results.

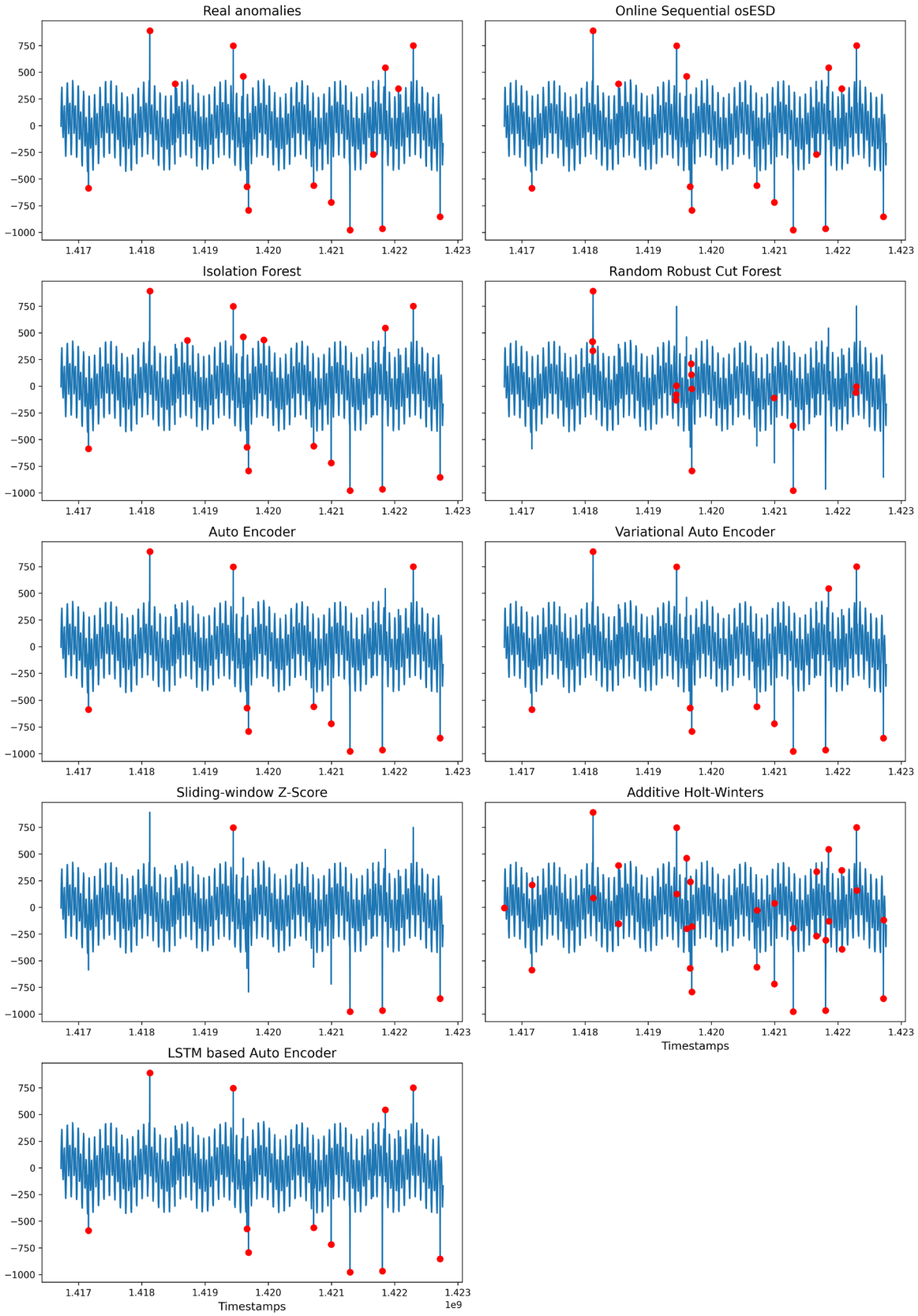


Fig. 4 Result plots of comparison tests on sample A3Benchmark-TS77

4 Experiments

In this section, we provide the experimental results of the proposed model and performance comparisons with existing methods using several datasets. We ran all tests with the specifications described in Table 7.

We used three types of datasets: 'ARIMA' datasets, 'seasonal' datasets, and 'Yahoo! Benchmark' [17] datasets. The first two were synthetically created, and the theory and methods used in creating these datasets can be found in Section 3. The Yahoo! datasets are divided into four parts: 'A1Benchmark', 'A2Benchmark', 'A3Benchmark' and 'A4Benchmark'. 'A1Benchmark' datasets are real-traffic datasets, and the remaining are synthetic datasets created by Yahoo!. In short, 96 'ARIMA' datasets, 100 'seasonal' datasets, 67 'A1Benchmark', 100 'A2Benchmark', 100 'A3Benchmark', and 100 'A4Benchmark' were used in testing and each respective score is displayed below. All datasets were labeled. Figure 3 shows examples of the final datasets with anomalies included. All codes related to synthetic dataset creation and anomaly generation are in the DATA.R file. To reproduce the test results, run RUN-TESTS.PY in PYTHON and RUN-TESTS.R in R and test results should appear in each TEST-RESULTS file respectively.

Table 3 Test results of ablation tests

Measure	**osESD**	oGESD	sosESD	norep_osESD
(a) Total				
Recall	0.787	**0.846**	0.659	0.349
Precision	**0.857**	0.431	0.447	0.830
F1-score	**0.795**	0.528	0.490	0.473
Time (sec)	0.410	0.583	**0.125**	0.489
(b) A1Benchmark				
Recall	0.334	0.311	**0.510**	0.288
Precision	**0.546**	0.176	0.467	0.505
F1-score	0.357	0.186	**0.403**	0.309
Time (sec)	0.111	0.127	**0.026**	0.131
(c) A2Benchmark				
Recall	0.902	**0.935**	0.673	0.390
Precision	**0.929**	0.520	0.611	0.836
F1-score	**0.905**	0.620	0.624	0.512
Time (sec)	0.232	0.311	**0.061**	0.236
(d) A3Benchmark				
Recall	0.822	**0.910**	0.460	0.390
Precision	0.892	0.604	0.077	**0.898**
F1-score	**0.847**	0.707	0.126	0.537
Time (sec)	0.139	0.180	**0.029**	0.153
(e) A4Benchmark				
Recall	0.589	**0.811**	0.329	0.344
Precision	0.858	0.614	0.107	**0.866**
F1-score	0.656	**0.676**	0.141	0.480
Time (sec)	0.139	0.160	**0.031**	0.161
(f) ARIMA dataset				
Recall	**0.968**	**0.968**	0.964	0.258
Precision	**0.954**	0.170	0.784	0.923
F1-score	**0.960**	0.288	0.859	0.402
Time (sec)	1.039	1.739	**0.364**	1.342
(g) Seasonal state-space				
Recall	0.967	0.971	**0.979**	0.402
Precision	**0.863**	0.405	0.654	0.850
F1-score	**0.907**	0.571	0.772	0.539
Time (sec)	0.835	1.046	**0.246**	0.933

We now present test scores. We show our test results with various plots for reference. Since the anomaly detection task is a binary classification problem, our main metrics are recall, precision, F1-score and Type I error, as follows:

$$Recall = \frac{(True\ Positive)}{(True\ Positive) + (False\ Negative)}, \tag{13}$$

$$Precision = \frac{(True\ Positive)}{(True\ Positive) + (False\ Positive)}, \tag{14}$$

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}, \tag{15}$$

$$Type\ I\ Error = \frac{(False\ Positive)}{(False\ Positive) + (True\ Negative)}, \tag{16}$$

where *True Positive* is the number of correctly classified positive points, *False Negative* is the number of wrongly classified negative points, *False Positive* is the number of wrongly classified positive points, and *True Negative* is the number of correctly classified negative points. Recall is synonymous with statistical power in statistics. F1-score is the harmonic mean of recall and precision, providing a single value that considers the trade-off between these two values. Type I error, also known as the false positive rate, represents the proportion of normal points that are incorrectly identified as anomalies. Since these four metrics are often used in anomaly detection, we choose them as our main metrics. We also include the execution times of the tested models.

We organize the results of our tests: Section 4.1 shows our proposed osESD against other models with the same datasets, Section 4.2 shows the ablation test results of our method, Section 4.3 shows the results of our expanded AUTO-OSESD method, and Sect. 4.4 shows the sensitivity analysis for injection strategies.

4.1 Comparison with other models

To compare our suggested osESD model with other models, we chose seven popular models ranging from statistical methods and machine learning to deep learning. These are the Sliding-window Z-score model (Z-Score), the additive Holt-Winters model (HW-A) [9, 22], isolation forest (IF) [15], the robust random cut forest (RRCF) [6],

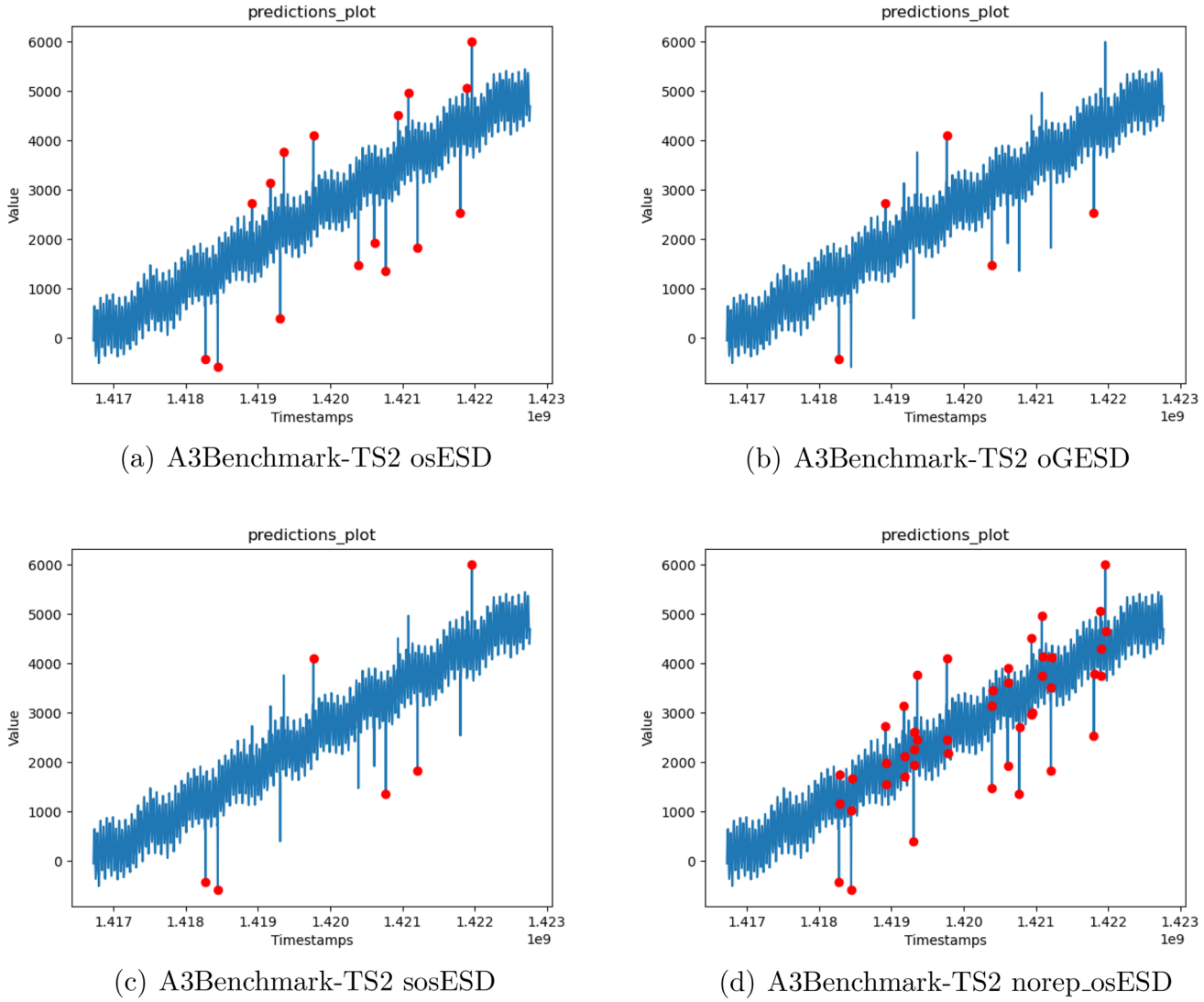


Fig. 5 A3Benchmark-TS2 ablation test results

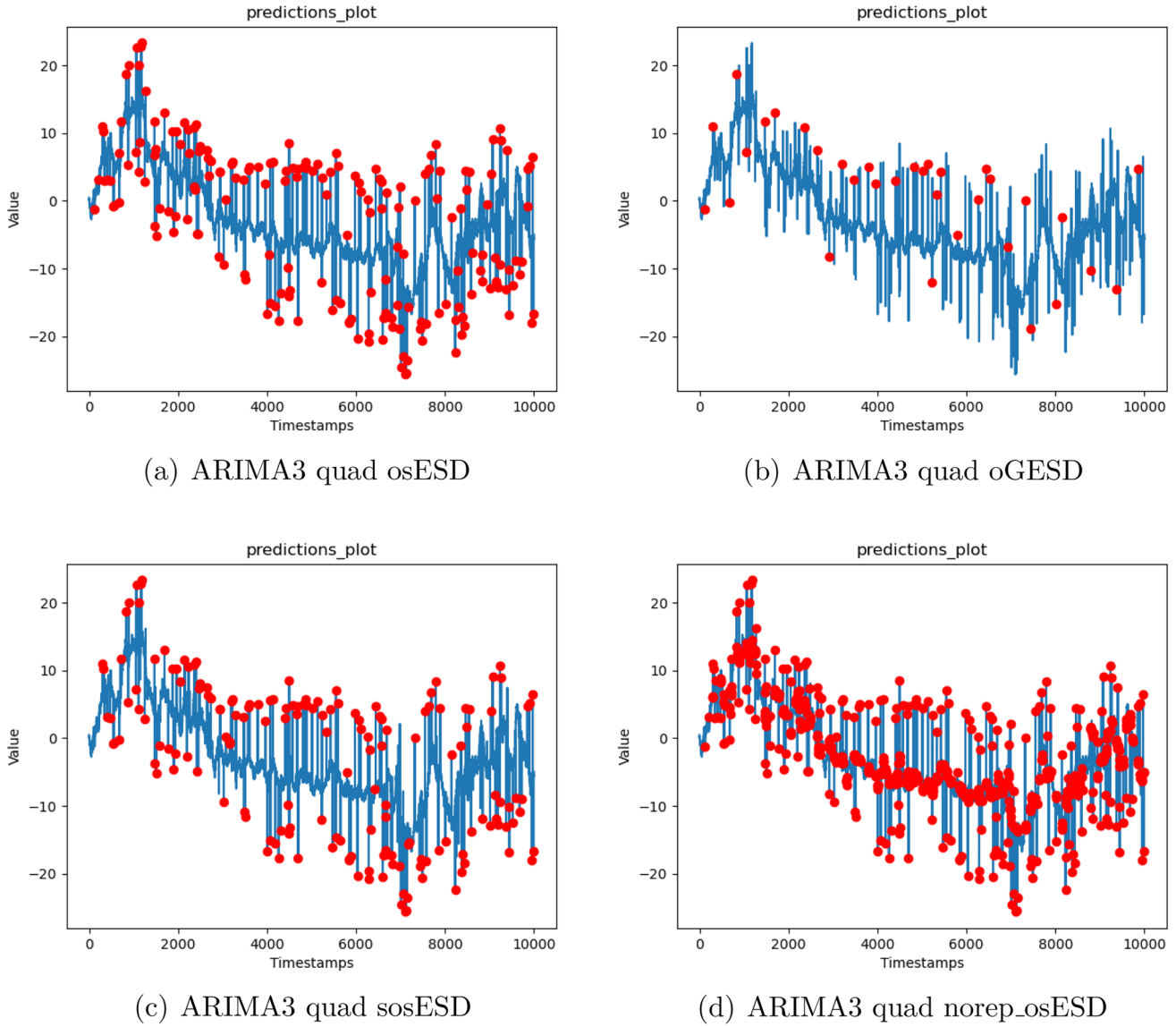
the auto-encoder model (AE) [7, 13], the variational auto-encoder (VAE) [4], and the LSTM encoder-decoder model (LSTM-ED) [8].

With eight models, including our proposed osESD, we conducted anomaly detection on the same datasets. Table 2 shows the results produced from `Python` codes. Figure 4 also shows detected anomalies by model along with the ground-truth anomalies on the Yahoo! A3Benchmark-TS77 dataset.

Table 2 shows averaged performance measures for the datasets. Bold-faced numbers represent the best values for the cases. The topmost case 'Total' shows averaged measures for all datasets. Our osESD implementation shows the best recall, precision and F1-scores in most cases, particularly dominating in (a), (c), (d), (e), and (f). While LSTM-ED and IF performed better in the 'A1Benchmark' (b), and Z-Score and HW-A showed slightly higher F1-score and recall in the 'Seasonal state-space' (g), our method remains a highly competitive and accurate anomaly detection tool for univariate time series.

In terms of efficiency, although osESD is significantly faster than deep learning models and RRCF, its run time was longer than statistical methods like IF, HW-A, and especially Z-score, which recorded the fastest time in all cases. RRCF exhibited prolonged execution times and yielded unsatisfactory outcomes, possibly attributable to the inherent variability in the time series data. The model dynamically updates its hyperplanes over time, but the initial characteristics may become outdated as the data evolves, causing distortions in the final predictions.

VAE consistently showed better results than AE, and LSTM-ED performed better than VAE in terms of F1-scores. Statistical models such as Z-Score and HW-A also showed strong performance in specific datasets like



(a) ARIMA3 quad osESD (b) ARIMA3 quad oGESD

(c) ARIMA3 quad sosESD (d) ARIMA3 quad norep_osESD

Fig. 6 ARIMA scenario 3 with quadratic anomalies dataset ablation test results

(f) and (g). However, in the overall average (Total), osESD outperformed all other models in terms of accuracy metrics. The primary reason for this phenomenon is the nature of our datasets. Being univariate, they might lack latent representations that deep learning methods can exploit effectively. Moreover, deep learning models typically rely on numerous layers and a large number of parameters, which often necessitates substantial amounts of data. Although our synthetic ARIMA and state-space datasets exhibit relatively longer data lengths (10,000 points), the Yahoo! benchmark datasets are considerably shorter, resulting in poorer performances of deep learning models. If the datasets were substantially longer or multivariate, these deep learning models might be able to use their advantages and demonstrate greater potential.

4.2 Ablation tests

We also conducted ablation tests to show that our contributions do indeed lead to better anomaly detection. We use a total of three modified models for ablation testing: simplified online sequential ESD (sosESD), online general ESD (oGESD), and osESD without regression replacement (norep-osESD). sosESD is a model where the two trend vectors are excluded, meaning the algorithm runs sequential ESD with the raw data. oGESD uses GESD tests instead of our proposed sequential ESD with the two trend vectors included, meaning it is incapable of effective online learning. norep-osESD uses sequential ESD and the two trend vectors, but does not replace X_{N+1} with the predicted value from regression $\hat{X}_{N+1}$ when it is deemed an anomaly. We

Table 4 Test results of AUTO OSESD results

Dataset	Windowsize	precision	Condition	recall	maxrF1-score	dwin	rwin	Alpha
ARIMA1_bernoulli1	50		And		3	2	5	0.05
	0.9756		**0.9346**		**0.9547**			
ARIMA1_uniform1	50		And		5	2	4	0.005
	0.8889		**0.9789**		**0.9323**			
ARIMA1_linear1	150		And		5	2	4	0.05
	0.8894		**0.9663**		**0.9263**			
ARIMA1_quadratic1	50		And		3	2	5	0.01
	0.9877		**1**		**0.9938**			
State-space_bernoulli1	50		And		3	5	4	0.01
	0.9381		**1**		**0.9681**			
State-space_uniform1	50		Or		3	10	30	0.01
	1		**0.9279**		**0.9626**			
State-space_linear1	50		And		3	2	10	0.001
	0.9741		**0.9947**		**0.9843**			
State-space_quadratic1	50		And		3	2	10	0.005
	0.898		**1**		**0.9462**			
APPL	118		Or		7	5	30	0.005
TSLA	64		And		3	30	5	0.05

will be comparing our proposed OSESD model with these three other models.

Table 3 shows the averaged performance measures for the tests. Tests were run in both `Python` and `R`, and both showed the exact same recall and precision results in all datasets. Runtimes are from the `R` implementations. Results can be reproduced by running RUN-ABLATION-TESTS.PY in `Python` or RUN-ABLATION-TESTS.R in `R` in the designated directory, and results will appear in the "TEST-RESULTS/ABLATION-TEST" file. Identical parameters were used for all ablation models and OSESD.

Bold-faced numbers represent the best values for each case. The top row 'Total' is the average value of all datasets combined. Our OSESD shows the best F1-scores compared to the other three in most cases, supporting the value of our contributions. In the two cases (b) and (d) when it was not the best, its scores were only slightly worse than the top score. In case (b) 'A1Benchmark dataset', SOSESD showed slightly higher F1-scores compared to our implementation. In cases (c) and (d), our implementation showed the best scores and in case (e), OGESD exhibited slightly higher F1-scores. In both ARIMA datasets and seasonal state-space model datasets, our implementation showed much better F1-scores than the other methods. In addition, due to its online learning capability, OSESD shows faster learning in all cases compared to OGESD. SOSESD exhibited the fastest learning times across all scenarios, which aligns with our expectations given its streamlined architecture. However, it displayed significant variation in F1-scores across different datasets.

Finally, OSESD consistently proved better than NOREP_OSESD, showing that our regression and replacement steps do indeed have benefits.

Some results of the detected anomalies are presented in Figs. 5 and 6. One can easily notice that our proposed OSESD implementation is the most accurate at finding anomalies.

4.3 Automated osESD

We also tested AUTO-OSESD, a package for automatically learning the OSESD parameters by grid search. We used the datasets described above, including two real-life unlabeled stock volume datasets. Both of these datasets are from Yahoo! finance, and represent stock volumes of Apple [25] and Tesla [26]. Apple stock prices were set as from January 1, 2001 to June 30, 2023 and Tesla from June 29, 2010 to June 30, 2023. This makes a total of 5926 datapoints for Apple and 3227 datapoints for Tesla. Figure 13 shows plots of both datasets.

Table 4 summarizes the performance results for the datasets used along with the parameters shown in Table 1. Test results can be reproduced by running RUN-AUTO-OSESD.PY in `Python` and RUN-AUTO-OSESD.R in `R`.

Our codes used a total of eight synthetic datasets and two real-life stock volume datasets. The Yahoo! benchmark datasets were not used because they are too short for reliable tuning. In addition, since both real-life stock volume datasets were unlabeled, we did not include their scores in Table 4. However, as mentioned in 3.5, pseudo y values were generated in the tuning process and the best parameters were found.

The evaluation of the datasets revealed consistently high F1-scores across all scenarios. Notably, several datasets exhibited a recall score of 1, a particularly advantageous outcome for real-world industrial anomaly detection

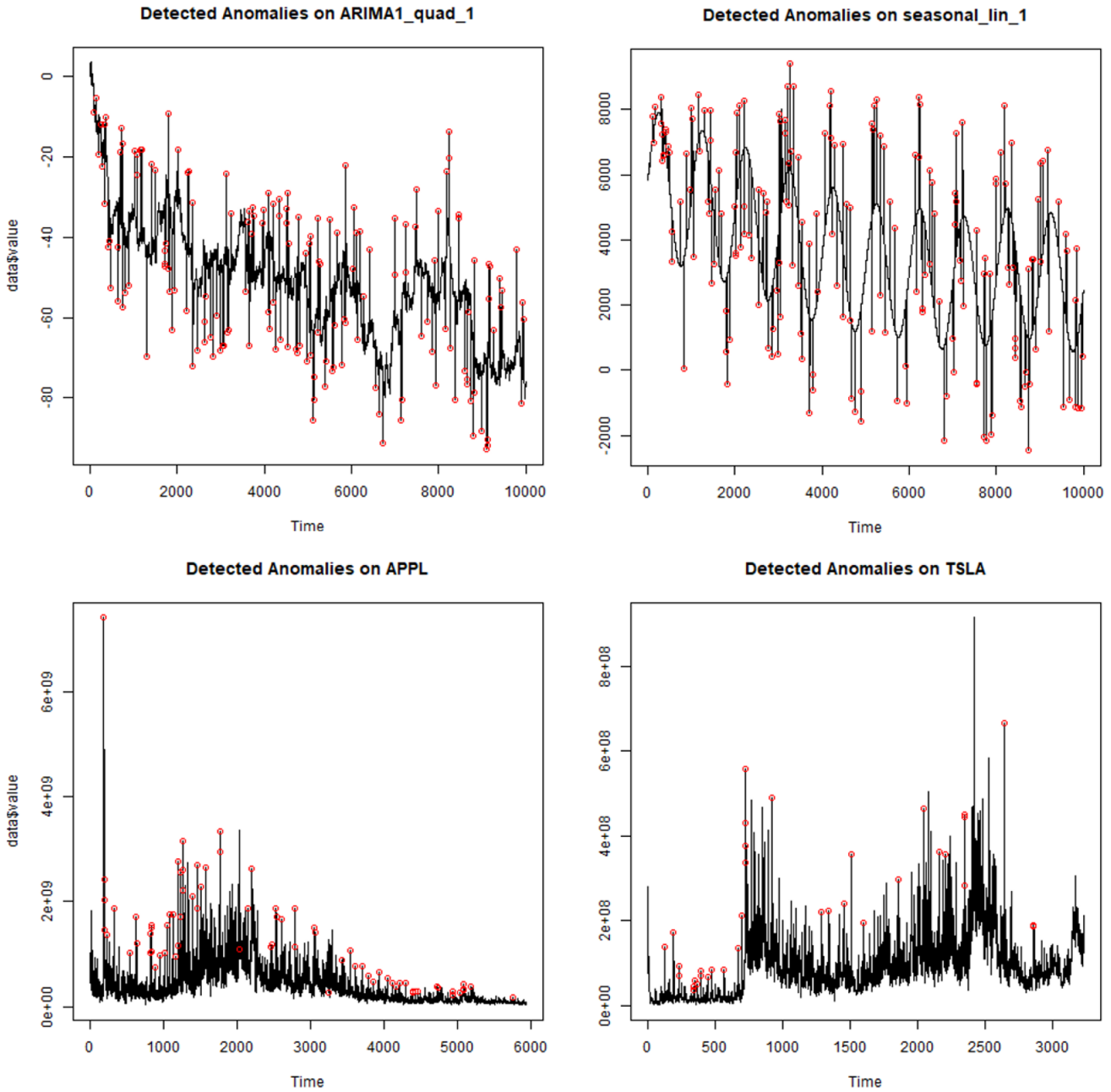


Fig. 7 Plots of AUTO OSESD results

challenges. Among the tested window sizes, a window size of 50 consistently yielded the most favorable results. Furthermore, when considering the 'condition' parameter, employing both vectors for anomaly assessment appeared to yield enhanced performance. This approach elevates the threshold for classifying a data point as an anomaly, which typically results in decreased recall rates. Nevertheless, as demonstrated in the results, achieving a recall score of 1 even under these conditions emphasizes the remarkable effectiveness of our approach. In its entirety, the AUTO-OSESD technique demonstrated exceptional performance and exhibited straightforward implementation across a diverse range of datasets. Figure 7 shows plots of anomalies predicted using AUTO-OSESD.

4.4 Sensitivity to injection strategy

We conducted an experiment to evaluate the sensitivity of detection performance to different injection strategies. To ensure a fair comparison, the parameter tuning for each strategy was performed using the exact search space defined in Table 6. By maintaining a consistent grid across all tests,

Table 5 Sensitivity of AUTO osESD performance to injection strategies

Dataset (Target)	Injection strategy	Precision	Recall	F1-score
ARIMA1 (Bernoulli)	Bernoulli	**0.9688**	0.9954	**0.9819**
	Drift	0.6781	**1.0000**	0.8082
	Block	0.6507	**1.0000**	0.7884
ARIMA1 (Linear)	Linear	**0.9617**	0.9853	**0.9734**
	Drift	0.8145	**0.9902**	0.8938
	Block	0.8016	**0.9902**	0.8860
State-space (Bernoulli)	Bernoulli	0.9854	0.9806	0.9830
	Drift	0.8929	0.9662	0.9281
	Block	**0.9855**	**0.9951**	**0.9903**
State-space (Linear)	Linear	**1.0000**	0.8043	0.8916
	Drift	0.9231	0.8571	0.8889
	Block	0.9345	**0.8626**	**0.8971**

we can confirm that any observed performance shifts are attributable to the injection pattern rather than to differences in the optimization process.

In this analysis, we compared the default Bernoulli and Linear distributions with manually implemented Drift and Block patterns. The drift pattern involves a steady increase in values over a window, while the block pattern applies a constant magnitude over a fixed duration. These strategies were tested against the actual anomalies in both the ARIMA and State-space datasets to determine how well each tuning method reflects the underlying data patterns.

Table 5 summarizes the results, where the parameters for each strategy were selected by maximizing the $F1$-score during the tuning phase. Overall, the detection performance remains effective across all strategies, though the specific values vary by dataset. In the ARIMA1 dataset, matching the injection strategy to the target anomaly pattern led to better results. Specifically, the Bernoulli and Linear patterns achieved $F1$-scores of 0.9819 and 0.9734 when aligned with their respective strategies. For the State-space dataset, the Block strategy showed that a strong signal can be effective for noisy and seasonal data, yielding the highest $F1$-scores of 0.9903 and 0.8971. These results indicate that while the AUTO-osESD maintains stable detection capability across different settings, selecting an injection strategy that reflects the anomaly characteristics is beneficial for achieving better results.

5 Conclusions

This paper introduces a novel approach called osESD for detecting anomalies in time series data with varying patterns. Our proposed method is implemented in both `Python` and `R`, providing versatile options for users. osESD builds upon the traditional GESD tests by incorporating change-rate trend vectors and residual trend vectors, along with an online learning mechanism. This combination allows for swift and accurate adaptation to changing data patterns.

By addressing the limitations associated with GESD, including the assumption of dataset normality and the lack of online learning capabilities, our enhanced approach is well-suited for the modern data landscape characterized by streams of data with varying trends.

In Section 4, we present detailed test results across a variety of synthetic and real-world datasets. These results show the performance of our algorithm when compared to other anomaly detection models. For the sake of convenience, we offer the AUTO-osESD tool in both `Python` and `R`. This tool automates the process of parameter learning based on an initial dataset, enabling anomaly detection within streaming data environments.

However, while our method improved F1-scores and reduced execution times, it currently lacks a formal mathematical proof for Type I error control and is limited to univariate datasets. In future research, we plan to develop a theoretical framework to formally state the conditions and assumptions under which osESD maintains its statistical properties. We also aim to make the method publicly available through platforms such as `PyPI` and `CRAN`.

Appendix

Python implementation

We ran all PYTHON tests in a virtual environment using PYTHON in PyCharm IDE. Some of the libraries included are NUMPY, PANDAS, MATPLOTLIB, and SCIPY which can all be easily imported.

We uploaded our package osESD to PyPI to a url link [11] that can be downloaded with the below code in command prompt:

```
cmd > pip install  osESD
```

Next, functions osESD and AUTO_osESD can be imported with the following:

```
>>> from osESD import osESD, auto_osESD
```

Below are example runs of osESD and AUTO osESD. First, we show a basic example with the Yahoo Benchmark dataset [17], which is often used for time series anomaly detection. The dataset can be utilized after downloading the data from the provided link.

```
>>> from osESD import osESD, auto_osESD
>>> from utils import scores_module
>>> import numpy as np
>>> import pandas as pd
```

```
>>> import matplotlib.pyplot as plt
>>> import os
>>> import time
>>>
>>> df = pd.read_csv("data//A2Benchmark_synthetic_44.csv")
>>> print(df.head())
```

The dataset, as in the following, is formed with VALUE as data value x_t, and ANOMALY as anomaly label 0 for normal and 1 for anomaly.

```
timestamps value anomaly
0 1416726000 567.318271 0
1 1416729600 -76.706404 0
2 1416733200 710.359907 0
3 1416736800 741.713439 0
4 1416740400 419.929424 0
```

The code for running a simple osESD example is as follows:

```
>>> anoms_1 = osESD(df, value_name='value',anomaly_name='anomaly')
>>> print(anoms_1)
```

When the above code is run, osESD will return a single list to ANOMS-1 as follows:

```
[623, 624, 625, 658, 659, 660, 893, 894, 895]
```

Codes for a full running example are provided in the following. We assume the libraries are already imported.

```
>>> df = pd.read_csv("data//A2Benchmark_synthetic_44.csv")
>>> size = 100
>>> labeled = True

>>> t1 = time.time()
>>> anoms_1 = osESD(df, value_name='value',anomaly_name='anomaly',size=size)
>>> t2 = time.time()

>>> dir_path = "osESD_Py_results/"
>>> if not os.path.exists(dir_path):
        os.makedirs(dir_path)
        print("Directory created:", dir_path)
>>> else:
        print("Directory already exists:", dir_path)


>>> if labeled:
        true_outlier = df['anomaly'][(size + 1):].astype(int)
        pred_outlier = pd.Series(0, index=true_outlier.index)
        pred_outlier.loc[anoms_1] = 1
        values = scores_module.return_PRF_values(true_outlier, pred_outlier, t1, t2)
        output = f"\nPrecision : {values[2]}, Recall : {values[3]}, F1-score : {values[4]}, " \
                f"Time : {values[5]} sec\n\n" \
                f"Real : [ {', '.join(map(str, df[df['anomaly'] == 1].index))} ]\n\n" \
                f"Anomalies : [ {', '.join(map(str, anoms_1))} ]"
        file_path = os.path.join(dir_path, "A2_benchmark_synthetic_44_osESD_result.txt")
        with open(file_path, 'w') as f:
            f.write(output)
        print(output)

>>> if True:
        plt.figure()
        plt.plot(df['value'], label='Data')
        plt.scatter(anoms_1, df.loc[anoms_1, 'value'], color='red', label='Anomalies')
        plt.title("Detected Anomalies on A2_benchmark_synthetic_44 with osESD")
        plt.legend()
        plt.savefig(os.path.join(dir_path, "A2_benchmark_synthetic_44_osESD_result.png"))
        plt.close()
```

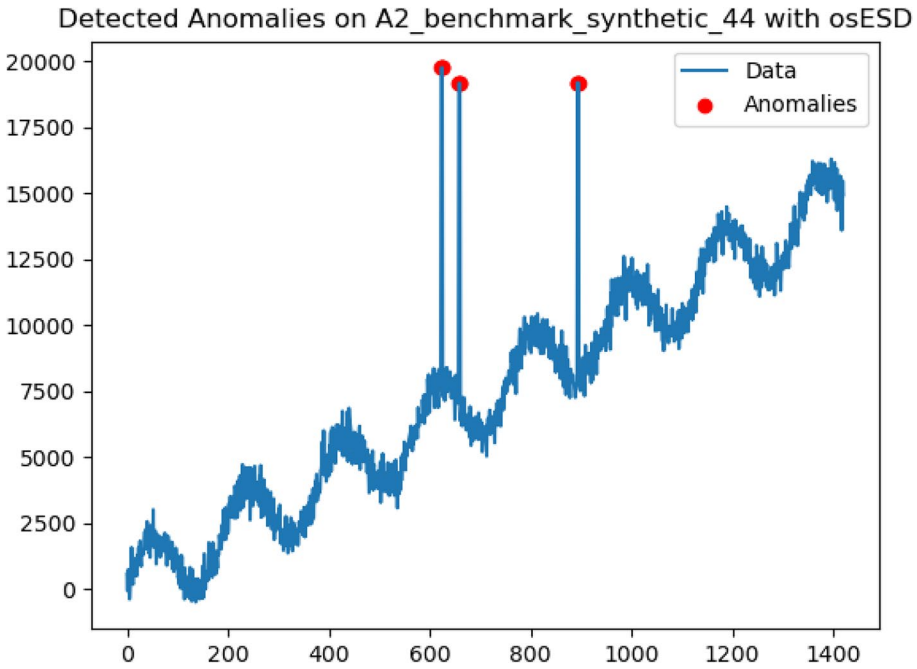


Fig. 8 Results of the Python osESD test in the BenchmarkA2 dataset

This will create a text output and a MATPLOTLIB plot in the osESD-PY-RESULTS directory, provided below in Fig. 8.

```
Precision : 1.0, Recall : 1.0, F1-score : 1.0, Time : 0.718 sec
Real : [623, 624, 625, 658, 659, 660, 893, 894, 895]
Anomalies : [623, 624, 625, 658, 659, 660, 893, 894, 895]
```

Another example on a seasonal dataset is provided as below. In this example, the parameters are adjusted in the osESD function.

```
>>> df = pd.read_csv("data//seasonal_ber_21.csv")
>>> labeled = True
>>> t1 = time.time()
>>> anoms_2 = osESD(df, value_name='value',anomaly_name='anomaly',size=100, dwin=5, rwin=5,
                                                    maxr=6, alpha=0.01)
>>> t2 = time.time()

>>> if labeled:
        true_outlier = df['anomaly'][(size + 1):].astype(int)
        pred_outlier = pd.Series(0, index=true_outlier.index)
        pred_outlier.loc[anoms_2] = 1
        values = scores_module.return_PRF_values(true_outlier, pred_outlier, t1, t2)
        output = f"\nPrecision : {values[2]}, Recall : {values[3]}, F1-score : {values[4]}, " \
                f"Time : {values[5]} sec\n\n" \
                f"Real : [ {', '.join(map(str, df[df['anomaly'] == 1].index))} ]\n\n" \
                f"Anomalies : [ {', '.join(map(str, anoms_2))} ]"
        file_path = os.path.join(dir_path, "seasonal_ber_21_osESD_result.txt")
        with open(file_path, 'w') as f:
            f.write(output)
        print(output)

>>> if True:

        plt.figure(figsize=(10, 10))  # Set the figure size to 10x10 inches
        plt.plot(df['value'], label='Data')
        plt.scatter(anoms_2, df.loc[anoms_2, 'value'], color='red', label='Anomalies')
        plt.title("Detected Anomalies on seasonal_ber_21 with osESD")
        plt.legend()
        plt.savefig(os.path.join(dir_path, "seasonal_ber_21_osESD_result.png"), dpi=1200)
        plt.close()
```

Its text output is shown in the following, and the figure in Fig. 9.

```
Precision : 1.0, Recall : 1.0, F1-score : 1.0, Time : 5.776 sec
Real : [137, 153, 214, 262, 289, 374,..., 9992, 9994]
Anomalies : [137, 153, 214, 262, 289, 374,..., 9992, 9994]
```

Our automated AUTO_OSESD can also be used after importing OSESD with FROM OSESD IMPORT OSESD, AUTO_OSESD. As for parameters, DATA is an input dataset, and WEIGHTS are relative weights for [Precision, Recall, F1-score, time]. If recall has the full weight, then a weight such as [0, 1, 0, 0] should be used. If recall and f1-score must be equally considered, then [0, 1, 1, 0] is suitable. If time and f1-score are the main metric, then WEIGHTS [0, 0, 1, 1] will return the best parameters for that case. PAR_LEN is length percentage $\mathcal{P}$ for grid searching. In all tests this is set to 20%.

If PARAMETERS are provided, AUTO_OSESD will search for all possible cases as specified in PARAMETERS. If not provided, it will use the basic parameter ranges shown in Table 6. Finally, MIN_MAX_SWITCH can be either 'True' or 'False', and when it is 'True', the function will interpret PARAMETERS as endpoints of the parameters. Thus, it will test equally spaced values between the two endpoints, and users only need to specify the start and end points. If specified as 'False', then the implementation will use only the values provided, whether those are the basic parameters or parameters provided by the user.

An example with a minimal parameter is as follows:

```
>>> df = pd.read_csv("data/ARIMA3_lin_4.csv")
>>> labeled = True
>>> t1 = time.time()
>>> auto_anoms_1 = auto_osESD(df)
>>> t2 = time.time()

>>> if labeled:
        true_outlier = df['anomaly'].astype(int)
        pred_outlier = [1 if i in auto_anoms_1 else 0 for i in range(len(df))]
        values = scores_module.return_PRF_values(true_outlier, pred_outlier, t1, t2)
        output = f"\nPrecision : {values[2]}, Recall : {values[3]}, F1-score : {values[4]}, " \
                 f"Time : {values[5]} sec\n\n" \
                 f"Real : [ {', '.join(map(str, df[df['anomaly'] == 1].index))} ]\n\n" \
                 f"Anomalies : [ {', '.join(map(str, auto_anoms_1))} ]"

        file_path = os.path.join(dir_path, "ARIMA3_lin_4_auto_osESD_result.txt")
        with open(file_path, 'w') as f:
            f.write(output)
        print(output)

>>> if True:
        plt.figure(figsize=(10, 10))  # Set the figure size to 10x10 inches
        plt.plot(df['value'], label='Data')
        plt.scatter(auto_anoms_1, df.loc[auto_anoms_1, 'value'], color='red', label='Anomalies')
        plt.title("Detected Anomalies on ARIMA3_lin_4 with auto_osESD")
        plt.legend()
        plt.savefig(os.path.join(dir_path, "ARIMA3_lin_4_auto_osESD_result.png"), dpi=1200)
        plt.close()
```

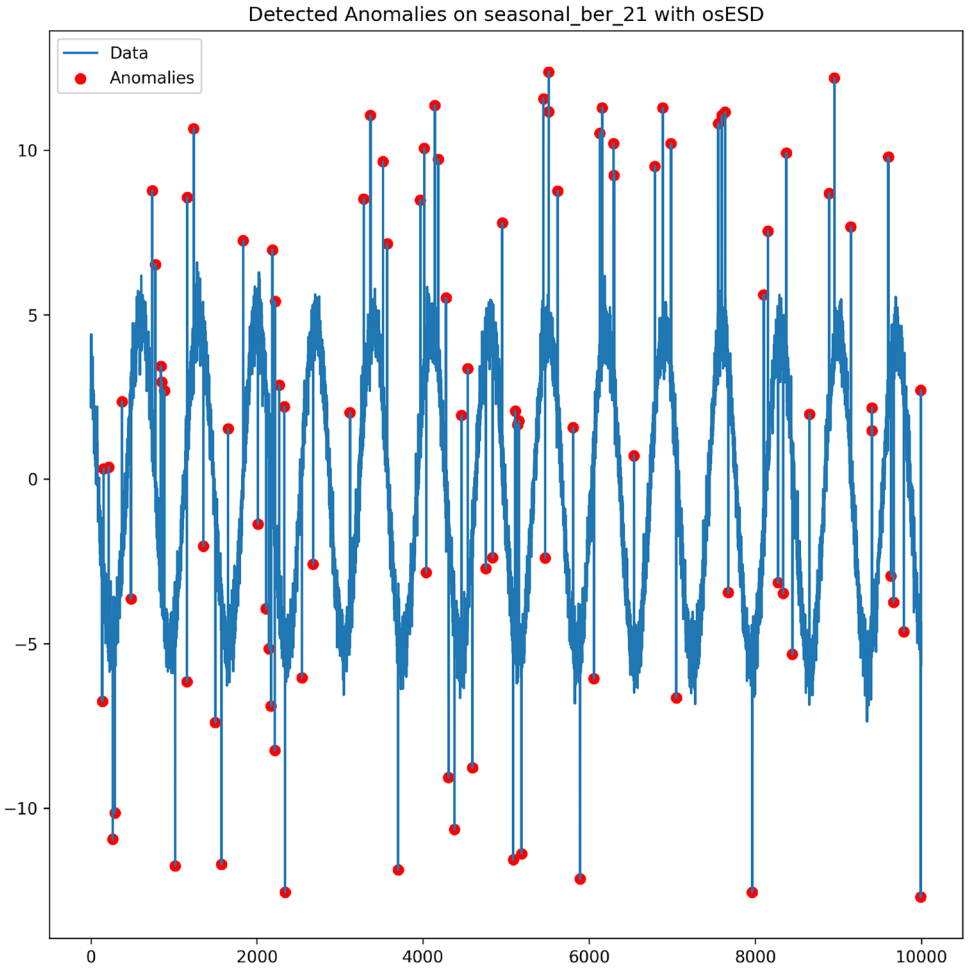


Fig. 9 Results of the python osESD test in the seasonal dataset

Then, after tuning and finding the best parameters, results of both text and plot are similar as the original osESD method:

```
No parameters detected, will use basic default for tuning.
Process : [1/2048]
Process : [512/2048]
Process : [1024/2048]
Process : [1536/2048]
Process : [2048/2048]
Now doing second step.
Precision : 1.0, Recall : 0.9707317073170731, F1-score : 0.9851485148514851, Time : 3478.699 sec
Real : [33, 49, 97, 99, 200, 279, 300, 340,..., 9965, 9971]
Anomalies : [97, 99, 200, 279, 300, 340,..., 9965, 9971]
```

Next, users can easily modify parameters as shown below. The below example modified all six parameters (sizes, conditions, alphas, dwins, rwins, and weights), setting learning length $\mathcal{P}$ to 0.2.

```
>>> df = pd.read_csv("data//seasonal_ber_21.csv")
>>> TEST_LEN = len(df)
>>> df = df.iloc[:TEST_LEN]
>>> size  = 100
>>> labeled = True
>>> t1 = time.time()
>>> auto_anoms_2 = auto_osESD(df, sizes=[50,200],conditions=[True,False], \
      alphas=[0.001,0.05],dwins=[2,30],rwins=[4,30],weights=[0,0,1,0],learning_length=0.2)
>>> t2 = time.time()

>>> if labeled:
        true_outlier = df['anomaly'].astype(int)
        pred_outlier = [1 if i in auto_anoms_2 else 0 for i in range(len(df))]
        values = scores_module.return_PRF_values(true_outlier, pred_outlier, t1, t2)
        output = f"\nPrecision : {values[2]}, Recall : {values[3]}, F1-score : {values[4]}, " \
                f"Time : {values[5]} sec\n\n" \
                f"Real : [ {', '.join(map(str, df[df['anomaly'] == 1].index))} ]\n\n" \
                f"Anomalies : [ {', '.join(map(str, auto_anoms_2))} ]"
        file_path = os.path.join(dir_path, "seasonal_ber_21_auto_osESD_result.txt")
        with open(file_path, 'w') as f:
            f.write(output)
        print(output)

>>> if True:
        plt.figure(figsize=(10, 10))  # Set the figure size to 10x10 inches
        plt.plot(df['value'], label='Data')
        plt.scatter(auto_anoms_2, df.loc[auto_anoms_2, 'value'], color='red', label='Anomalies')
        plt.title("Detected Anomalies on seasonal_ber_21 with auto_osESD")
        plt.legend()
        plt.savefig(os.path.join(dir_path, "seasonal_ber_21_auto_osESD_result.png"), dpi=1200)
        plt.close()
```

The results are as follows:

```
Will use suggested parameters.
Process : [1/128]
Process : [32/128]
Process : [64/128]
Process : [96/128]
Process : [128/128]
Now doing second step.
Precision : 1.0, Recall : 0.98989898989899, F1-score : 0.9949238578680203, Time : 245.853 sec
Real : [77, 137, 153, 214, 262, 289, 374,..., 9992, 9994]
Anomalies : [137, 153, 214, 262, 289, 374,..., 9992, 9994]
```

Table 6 Basic parameters of AUTO osESD

Parameter	Values
Window size	50,100,150,200
Choice condition	And, Or
maxr	3,5,7,10
dwin	2,5,10,30
rwin	4,5,10,30
Alpha	0.001,0.005,0.01,0.05

Finally, if the input dataset is unlabeled with no anomalies, the LABELED parameter should be set to be FALSE. This will produce plots and detect anomaly indices without performance scores:

```
>>> df = pd.read_csv("data//ARIMA5_quad_2.csv")
>>> TEST_LEN = len(df)
>>> df = df.iloc[:TEST_LEN]
>>> labeled = False
>>> t1 = time.time()
>>> auto_anoms_3 = auto_osESD(df, labeled=labeled)
>>> t2 = time.time()

>>> if labeled:
        true_outlier = df['anomaly'].astype(int)
        pred_outlier = [1 if i in auto_anoms_3 else 0 for i in range(len(df))]
        values = scores_module.return_PRF_values(true_outlier, pred_outlier, t1, t2)
        output = f"\nPrecision : {values[2]}, Recall : {values[3]}, F1-score : {values[4]}, " \
                f"Time : {values[5]} sec\n\n" \
                f"Real : [ {', '.join(map(str, df[df['anomaly'] == 1].index))} ]\n\n" \
                f"Anomalies : [ {', '.join(map(str, auto_anoms_3))} ]"
        file_path = os.path.join(dir_path, "ARIMA5_quad_2_auto_osESD_result.txt")
        with open(file_path, 'w') as f:
            f.write(output)
        print(output)
>>> else:
        pred_outlier = [1 if i in auto_anoms_3 else 0 for i in range(len(df))]
        output = f"Anomalies : [ {', '.join(map(str, auto_anoms_3))} ]"
        file_path = os.path.join(dir_path, "ARIMA5_quad_2_auto_osESD_result.txt")
        with open(file_path, 'w') as f:
            f.write(output)
        print(output)

>>> if True:
        plt.figure(figsize=(10, 10))  # Set the figure size to 10x10 inches
        plt.plot(df['value'], label='Data')
        plt.scatter(auto_anoms_3, df.loc[auto_anoms_3, 'value'], color='red', label='Anomalies')
        plt.title("Detected Anomalies on ARIMA5_quad_2 with auto_osESD")
        plt.legend()
        plt.savefig(os.path.join(dir_path, "ARIMA5_quad_2_auto_osESD_result.png"), dpi=1200)
        plt.close()
```

The results are as the following:

`Anomalies : [304, 312, 318, 330, 338, 474,..., 9894, 9987]`

R Implementation

We used R in |ver. 4.3.1 (2023-06-16 ucrt)| and RSTUDIO |ver. 2022.12.0+353|. One can download OSESD and AUTO_OSESD without difficulty by using Github as in the following R codes. The link for the Github repository is [10].

```
R> library(devtools)
R> install_github("iguv221/osESD",force=TRUE)
R> library(osESD)
```

The libraries, 'OGTPARSE', 'CARET', 'TCLTK', 'RETICULATE', and 'R6' will be installed with the loading of our package |library(osESD)|. Then, they can be loaded with the following:

|R> library(ogtparse);library(caret);|

|R> library(tcltk); library(reticulate); library(R6);|

Users can run our methods using either `osESD` or `auto_osESD` in R. Its testing datasets have been transformed and packaged in an RDA format.

A simple example of OSESD is presented below, starting with loading data.

```
R> data(A2Benchmark_synthetic_44)
R>input_data<-A2Benchmark_synthetic_44
R> labeled <- TRUE
R> size <- 100
R> anoms_1 <- osESD(data=input_data,size=size)
R> anoms_1
```

Then, the variable `anoms_1` will have a list of indices of anomalies. The output is the exact same as its PYTHON counterpart.

```
R> anoms_1
[1] 624 625 626 659 660 661 894 895 896
```

As above, a full running code for exporting results and plots can be executed as follows.

```
R> data_name <- "A2Benchmark_synthetic_44"
R> data(A2Benchmark_synthetic_44)
R> input_data <- A2Benchmark_synthetic_44
R> labeled <- TRUE
R> size <- 100

R> t1 <- Sys.time()
R> anoms_1 <- osESD(data=input_data,size=size)
R> t2 <- Sys.time()
R> anoms_1


R> dir_path <- "osESD_R_results//"
R> if (!file.exists(dir_path)) {
      dir.create(dir_path)
      cat("Directory created:", dir_path, "\n")
R> } else {
      cat("Directory already exists:", dir_path, "\n")
   }

R> if (labeled){
      true_outlier<-as.factor(input_data$anomaly[(size + 1):length(input_data$value)])
      pred_outlier <- rep(0, length(true_outlier))
      for (anom_index in anoms_1) {pred_outlier[anom_index] <- 1}
      pred_outlier <- as.factor(pred_outlier)
      values <- PRFTcalculator(true_outlier, pred_outlier, t1,t2)

      ### Write output text file if labeled.
      Output<-paste0("\nPrecision : ",values$precision,", Recall : ",values$recall,",
        F1-score : ",values$F1Score,", Time : ",values$Run_Time," (sec)\n\n",
        "Real : [ ", paste(as.character(which(input_data$anomaly==1)),collapse=", "), " ]\n\n",
        "Anomalies : [ ", paste(as.character(anoms_1),collapse=", "), " ]")
      file_path <- paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
                   "_osESD_result",".txt")
      writeLines(Output, file_path)
R> } else{
      ### Write output text file if unlabeled.
      Output<-paste0("\nAnomalies : [ ", paste(as.character(anoms_1),collapse=", "), " ]")
      file_path <- paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
                   "_osESD_result",".txt")
      writeLines(Output, file_path)
   }

R> if (TRUE){
      png(file=paste0(paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
             "_osESD_result",".png")), width=10, height=8, units="in", res=1200)
      plot.ts(input_data$value,
              main=sprintf(
              paste0("Detected Anomalies on ", gsub("//.csv", "", data_name)," with osESD")),
              xlab="Your X-Axis Label Here")  # Set the x-axis label
      points(anoms_1, input_data$value[anoms_1], col="red")
      dev.off()
    }

R> cat(Output)
```

Final line `R> cat(Output)` will generate the final results as the following:

```
  Precision : 1, Recall : 1, F1-score :
1, Time : 0.293712139129639 (sec)
  Real : [624, 625, 626, 659, 660, 661,
894, 895, 896]
  Anomalies : [624, 625, 626, 659, 660,
661, 894, 895, 896]
```

Similarly in Python, osESD in R can also adjuste its parameters accordingly. Such an example is as follows:

```
R> data_name <- "seasonal_ber_21"
R> data(seasonal_ber_21)
R> input_data <- seasonal_ber_21
R> labeled <- TRUE
R> size <- 100

R> t1 <- Sys.time()
R> anoms_2 <- osESD(data=input_data,size=size,dwin=5,rwin=5,maxr=6,alpha=0.01)
R> t2 <- Sys.time()
R> anoms_2

R> dir_path <- "osESD_R_results//"

R> if (!file.exists(dir_path)) {
      dir.create(dir_path)
      cat("Directory created:", dir_path, "\n")
R> } else {
      cat("Directory already exists:", dir_path, "\n")
   }

R> if (labeled){
      true_outlier<-as.factor(input_data$anomaly[(size + 1):length(input_data$value)])
      pred_outlier <- rep(0, length(true_outlier))
      for (anom_index in anoms_2) {pred_outlier[anom_index] <- 1}
      pred_outlier <- as.factor(pred_outlier)
      values <- PRFTcalculator(true_outlier, pred_outlier, t1,t2)

      ### Write output text file if labeled.
      Output<-paste0("\nPrecision : ",values$precision,", Recall : ",values$recall,",
        F1-score : ",values$F1Score,", Time : ",values$Run_Time," (sec)\n\n",
        "Real : [ ", paste(as.character(which(input_data$anomaly==1)),collapse=", "), " ]\n\n",
        "Anomalies : [ ", paste(as.character(anoms_2),collapse=", "), " ]")
      file_path <- paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
                "_osESD_result",".txt")
      writeLines(Output, file_path)
R> } else{
      ### Write output text file if unlabeled.
      Output<-paste0("\nAnomalies : [ ", paste(as.character(anoms_2),collapse=", "), " ]")
      file_path <- paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
                "_osESD_result",".txt")
      writeLines(Output, file_path)
    }

### Draw plot with anomalies.
R> if (TRUE){
      png(file=paste0(paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
          "_osESD_result",".png")), width=10, height=8, units="in", res=1200)
      plot.ts(input_data$value,
              main=sprintf(
              paste0("Detected Anomalies on ", gsub("//.csv", "", data_name)," with osESD")),
              xlab="Your X-Axis Label Here")  # Set the x-axis label
      points(anoms_2, input_data$value[anoms_2], col="red")
      dev.off()
}

R> cat(Output)
```

This generates the exact same results as the second example in PYTHON provided above.

```
  Precision : 1, Recall : 1, F1-score : 1, Time 2.20411 (sec)
  Real : [138, 154, 215, 263, 290, 375,..., 9993, 9995]
  Anomalies : [138, 154, 215, 263, 290, 375,..., 9993, 9995]
```

Table 7 Hardware specifications used in testing

Component	Specification
Processor	Intel(R) Xeon(R) w7-2475X Processor FC-LGA16A
Storage/RAM Memory	1 TB/64 GB
Graphics card	NVIDIA RTX4090
Operating system	Ubuntu

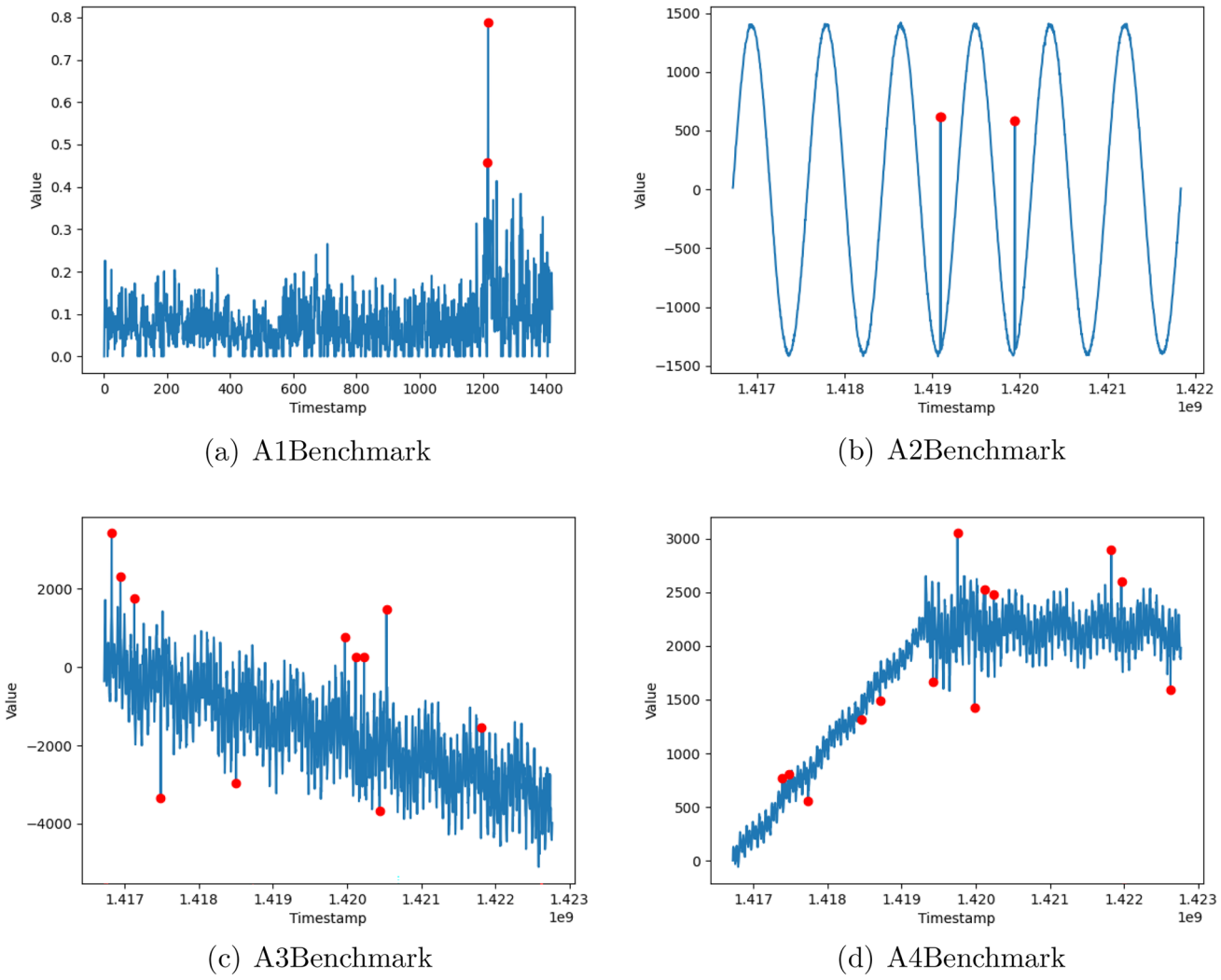
Function `auto_osESD` in R is also available. The function itself is equipped with a grid search for tuning parameters as explained above, so it does not need to manually find the best parameters for a specific dataset. The function will return the indices of anomalies in the dataset. A simple example is as follows:

```
R> data(ARIMA3_lin_4)
R> input_data <- ARIMA3_lin_4
R> auto_anoms_1_results<-auto_osESD(data=input_data,labeled=TRUE)
R> auto_anoms_1_results
```

It yields both the anomaly indices and the best parameters:

```
$pred_anoms_index
[1]  98  100  201  280  301...  9954  9966  9972
$params
[1] 0e+00 5e+01 3e+00 2e+00 4e+00 1e-03
```

Similarly, users can easily modify the input parameters of `auto_osESD` as in the PYTHON example.



(a) A1Benchmark (b) A2Benchmark

(c) A3Benchmark (d) A4Benchmark

Fig. 10 Plots of Yahoo! benchmark datasets

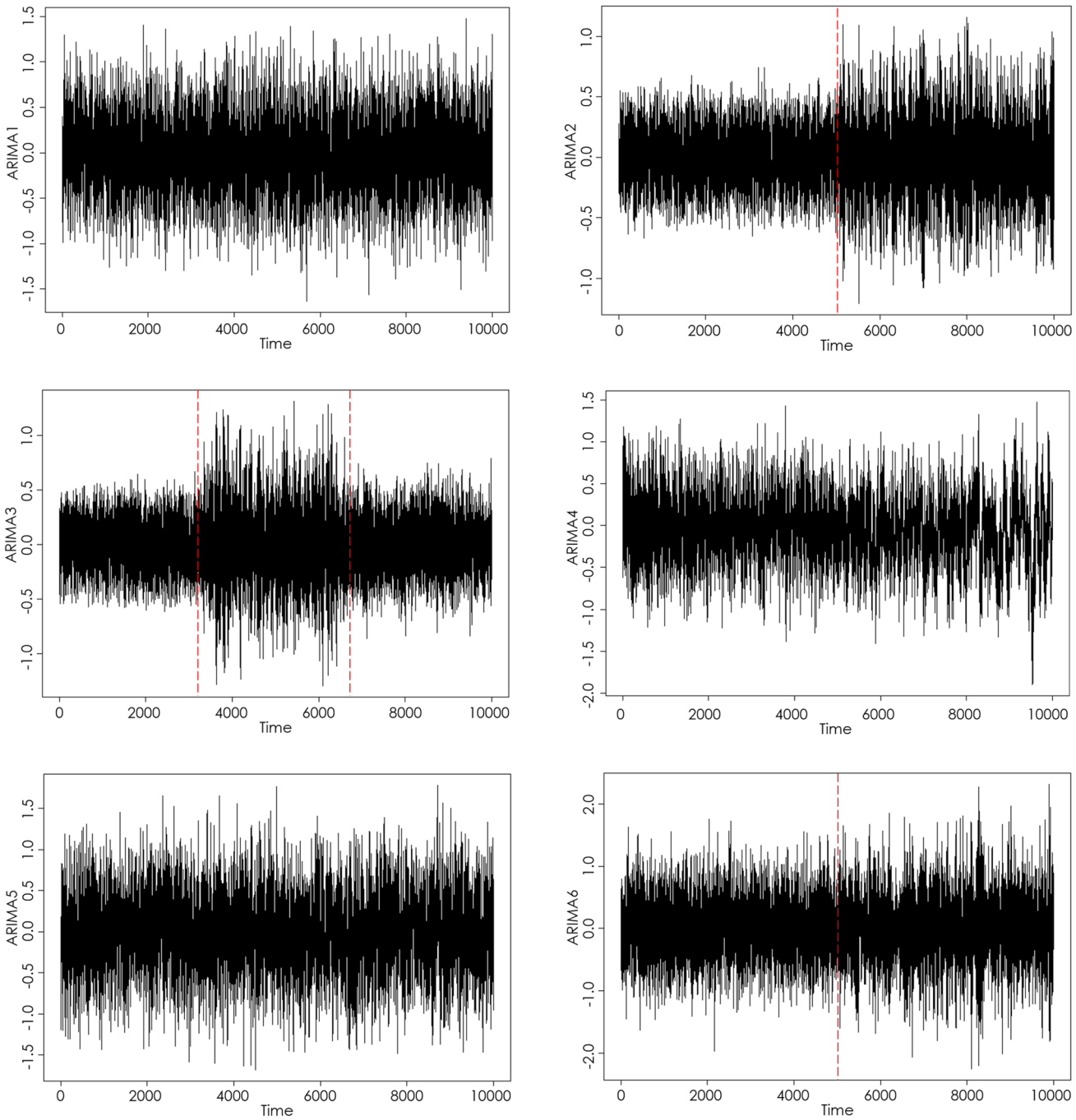


Fig. 11 ARIMA datasets scenarios 1 to 6

```
R> data_name <- "seasonal_ber_21"
R> data(seasonal_ber_21)
R> input_data <- seasonal_ber_21
R> labeled <- TRUE

R> dir_path <- "auto_osESD_R_results//"
R> if (!file.exists(dir_path)) {
      dir.create(dir_path)
      cat("Directory created:", dir_path, "\n")
R> } else {
      cat("Directory already exists:", dir_path, "\n")
}

R> ex_opt_list <- list(
      make_option(c("--WindowSizes"), type="double", default=c(50,200)),
      make_option(c("--AndOr"), type="double", default=c(1,0)),
      make_option(c("--MaxRs"), type="double", default=c(3,20)),
      make_option(c("--Dwins"), type="double", default=c(2,30)),
      make_option(c("--Rwins"), type="double", default=c(4,30)),
      make_option(c("--Alphas"), type="double", default=c(0.001,0.05)))
R> ex_opt <- parse_args(OptionParser(option_list = ex_opt_list))

R> labeled<-TRUE
R> min_max_switch<-TRUE
R> ex_weights<-c(0,0,1,0.1)

R> t1<-Sys.time()
R> auto_anoms_2_results<-auto_osESD(data=input_data,labeled=labeled,parameters=ex_opt,
       weights=ex_weights,min_max_switch = min_max_switch)
R> t2<-Sys.time()

R> auto_anoms_2<-auto_anoms_2_results$pred_anoms_index
R> params_2<-auto_anoms_2_results$params

R> if (labeled){
      true_outlier<-as.factor(input_data$anomaly[1:length(input_data$value)])
      pred_outlier <- rep(0, length(true_outlier))
      for (anom_index in auto_anoms_2) {pred_outlier[anom_index] <- 1}
      pred_outlier <- as.factor(pred_outlier)
      values <- PRFTcalculator(true_outlier, pred_outlier, t1,t2)

      ### Write output text file if labeled.
      Output<-paste0("\nPrecision : ",values$precision,", Recall : ",values$recall,",
      F1-score : ",values$F1Score,", Time : ",values$Run_Time," (sec)\n\n",
        "Real : [ ", paste(as.character(which(input_data$anomaly==1)),collapse=", "), " ]\n\n",
        "Anomalies : [ ", paste(as.character(auto_anoms_2),collapse=", "), " ]")
      file_path <- paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
                    "_auto_osESD_result",".txt")
      writeLines(Output, file_path)
R> } else{
      Output<-paste0("\nAnomalies : [ ", paste(as.character(auto_anoms_2),collapse=", "), " ]")
      file_path <- paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
                    "_auto_osESD_result",".txt")
      writeLines(Output, file_path)
}

R> cat(Output)
```

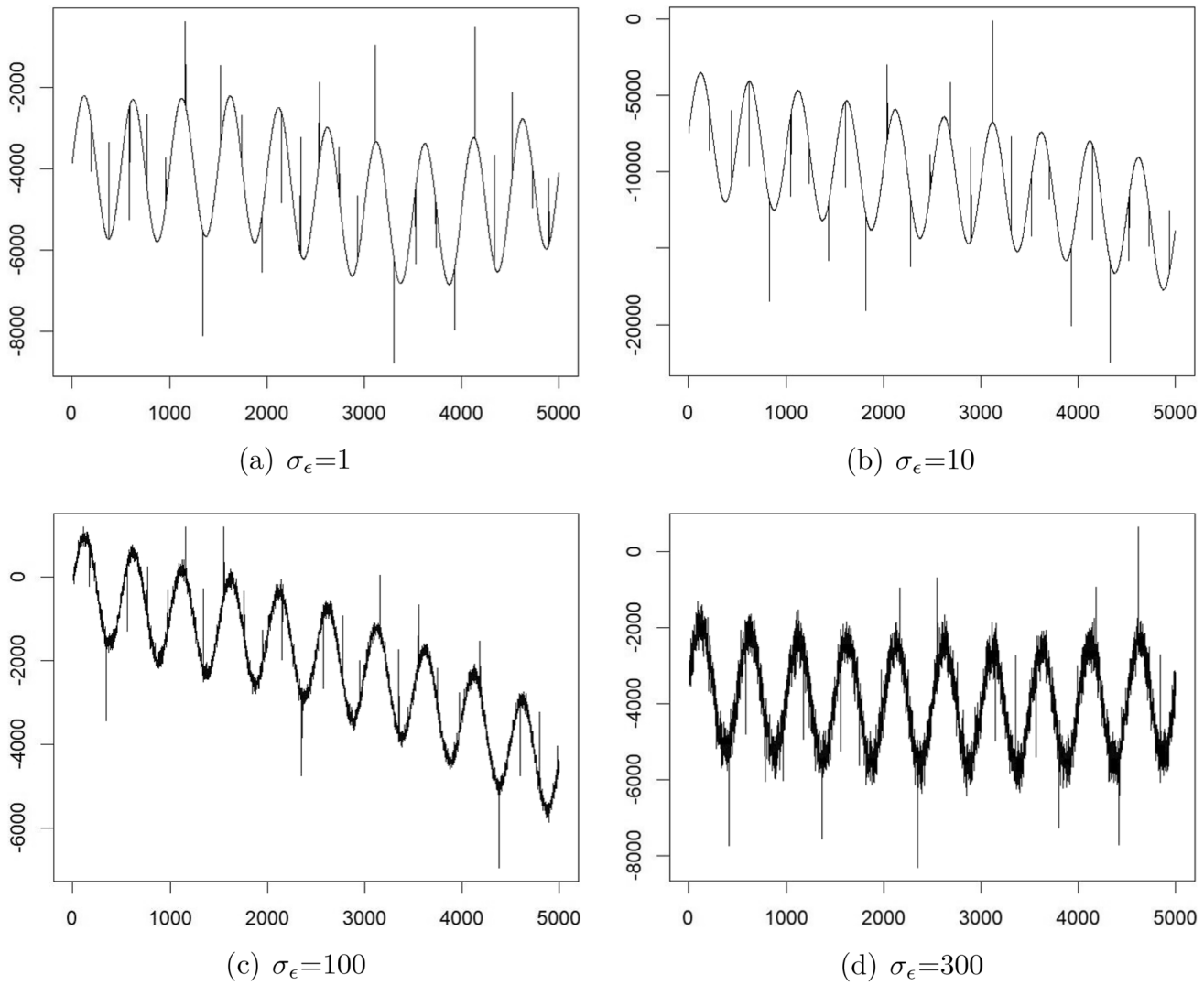


Fig. 12 Synthetic data with various σ_ϵ values

This produces the results below. Again, it shows identical results to the second `auto_osESD` example in PYTHON.

```
Precision : 1, Recall : 0.98989898989899,
F1-score : 0.99492385786802, Time :
3123.46700310707 (sec)s
Real : [78, 138, 154, 215, 263,...,
9791, 9993, 9995]
Anomalies : [138, 154, 215, 263,...,
9791, 9993, 9995]
```

To enable automatic parameter tuning of an unlabeled dataset, one simply has to change LABELED to FALSE, as in the following:

```
R> data_name <- "seasonal_ber_6"
R> data(seasonal_ber_6)
R> input_data <- seasonal_ber_6
R> labeled <- FALSE

R> dir_path <- "auto_osESD_R_results//"
R> if (!file.exists(dir_path)) {
      dir.create(dir_path)
      cat("Directory created:", dir_path, "\n")
R> } else {
      cat("Directory already exists:", dir_path, "\n")
}

R> t1<-Sys.time()
R> auto_anoms_3_results<-auto_osESD(data=input_data,labeled=labeled)
R> t2<-Sys.time()

R> auto_anoms_3<-auto_anoms_3_results$pred_anoms_index
R> params_3<-auto_anoms_3_results$params

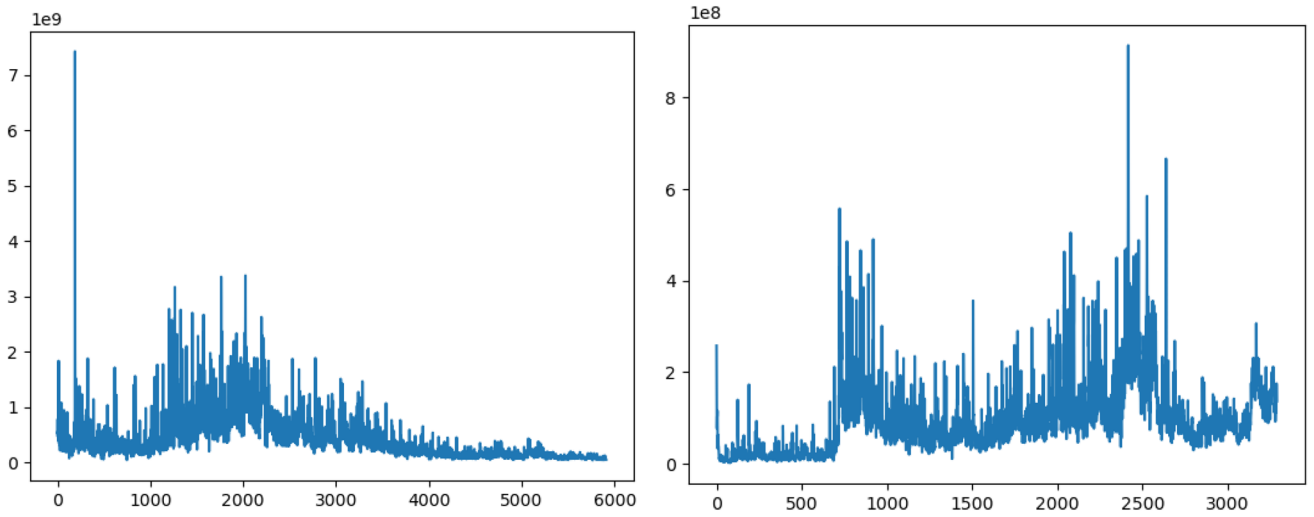
R> if (labeled){
      true_outlier<-as.factor(input_data$anomaly[1:length(input_data$value)])
      pred_outlier <- rep(0, length(true_outlier))
      for (anom_index in auto_anoms_3) {pred_outlier[anom_index] <- 1}
      pred_outlier <- as.factor(pred_outlier)
      values <- PRFTcalculator(true_outlier, pred_outlier, t1,t2)

      Output<-paste0("\nPrecision : ",values$precision,", Recall : ",values$recall,",
        F1-score : ",values$F1Score,", Time : ",values$Run_Time," (sec)\n\n",
        "Real : [ ", paste(as.character(which(input_data$anomaly==1)),collapse=", "), " ]\n\n",
        "Anomalies : [ ", paste(as.character(auto_anoms_3),collapse=", "), " ]")
      file_path <- paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
                   "_auto_osESD_result",".txt")
      writeLines(Output, file_path)
R> } else{

      Output<-paste0("\nAnomalies : [ ", paste(as.character(auto_anoms_3),collapse=", "), " ]")
      file_path <- paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
                   "_auto_osESD_result",".txt")
      writeLines(Output, file_path)
}

R> if (TRUE){
      png(file=paste0(paste0(Starting_point,"//",dir_path,gsub("//.csv", "", data_name),
                   "_auto_osESD_result",".png")),
          width=10, height=8, units="in", res=1200)
      plot.ts(input_data$value,
              main=sprintf(
              paste0("Detected Anomalies on ", gsub("//.csv", "", data_name)," with osESD")),
              xlab="Your X-Axis Label Here")  # Set the x-axis label
      points(auto_anoms_3, input_data$value[auto_anoms_3], col="red")
      dev.off()
}

R> cat(Output)
```



(a) APPLE stock volumes (b) TESLA stock volumes

Fig. 13 Apple and tesla stock volumes

The results are as the following:

```
Anomalies : [252, 349, 388,..., 9925, 9968]
```

Creating datasets

We utilized both synthetic and real-world datasets in our experiments. We included Yahoo! datasets, which contain one real-time and three synthetic datasets [17]. These are entitled 'A1Benchmark', 'A2Benchmark', 'A3Benchmark', and 'A4Benchmark' and 'A1Benchmark' pertains to real-life production traffic to some of the Yahoo! properties. The other three are synthetic datasets. All four datasets are labeled. Figure 10 illustrates some data of each dataset with red points being the labeled anomalies.

We also created two more types of synthetic datasets with anomalies added, each derived from an auto regressive integrated moving average (ARIMA) model and a seasonal state-space model. After creating a dataset, anomalies were generated and added to it.

The ARIMA synthetic datasets were created through an ARIMA model with different settings to show the robustness of our implementation in various situations. ARIMA datasets can be created with the following formula:

$$\nabla^d X_t = \sum_{i=1}^{q} \beta_i \epsilon_{t-1} + \sum_{i=1}^{k} \alpha_i \nabla^d X_{t-i} + \epsilon_t, \tag{17}$$

with α, β and d being main parameters in dataset creation and ϵ_t being random noise added to the dataset. Since the main purpose of this research is anomaly detection, and not time series forecasting itself, we set $d = 1$ in all scenarios since integration degrees hardly bring forth significant changes in the context of anomaly detection.

We created a total of six scenarios in the ARIMA dataset, as listed below. All ARIMA datasets had random noise ϵ_t following Gaussian distribution $\mathcal{N}(0, 0.2^2)$.

1. Sanity test: Simple sanity generates stationary datasets. The data generated here is a simple ARIMA-based time series with parameters $\alpha = [0.6, -0.5, 0.4, -0.4, 0.4]$ and $\beta = [0.3, -0.2]$. This test is to check which algorithm works best in the simplest and most fundamental of ARIMA datasets mimicking detrended real-life datasets.
2. One abrupt change: A non-stationary dataset with one abrupt change in parameters at the halfway point. We generate the first half of the dataset with $\alpha = [0.6, -0.5, 0.4, -0.4, 0.4]$ and $\beta = [0.3, -0.2]$, then the other half with $\alpha = [-0.4, -0.5, 0.4, 0.4, 0.1]$ and $\beta = [-0.3, 0.2]$. This represents abrupt changes in the time series dataset, and quick adaptation to change is both beneficial and crucial when either forecasting or classifying anomalies.
3. Two abrupt changes: Another non-stationary dataset with abrupt changes at the one-third and two-thirds points. In the first part the parameters are $\alpha = [0.6, -0.5, 0.4, -0.4, 0.4]$ and $\beta = [0.3, -0.2]$, and $\alpha = [-0.4, -0.5, 0.4, 0.4, 0.1]$ and $\beta = [-0.3, 0.2]$ for the second part. The third part returns to the same parameters as the first one. This scenario assumes situations in which the time series underwent abrupt but temporary changes and then returned to its normal parameters.
4. Gradual change: Instead of changing the parameters α and β abruptly, we changed α gradually in a linear fashion while leaving β untouched. We

changed α from $\alpha = [0.6, -0.5, 0.4, -0.4, 0.4]$ to $\alpha = [-0.4, -0.5, 0.4, 0.4, 0.1]$ linearly: in other words for time t, $\alpha_t = [0.6, -0.5, 0.4, -0.4, 0.4] \times (\frac{t}{N}) + [-0.4, -0.5, 0.4, 0.4, 0.1] \times (1 - \frac{t}{N})$. This represents ARIMA datasets that have parameters that continually change.

5. Correlated noise: We take into account cases where noise values are correlated with one another. This is to check scenarios where noise has some sort of unknown property. The noise correlation factor was set to 0.3, and parameters $\alpha = [0.6, -0.5, 0.4, -0.4, 0.4]$ and $\beta = [0.3, -0.2]$.
6. Correlated noise with one abrupt change: The final synthetic ARIMA dataset, being a combination of scenarios 2 and 5. We added correlated noise in the same manner with an abrupt change at the halfway point.Figure 11 shows plots of our six datasets before integration for easy comprehension. The actual datasets are values integrated by one degree using the values produced in the plots. Anomalies are then added to the integrated dataset.

Our second synthetic dataset, which is a state space model generated by a local linear trend model with seasonality, is as follows:

$$
\begin{aligned}
y_t &= \mu_t + \gamma_t + \epsilon_t, \quad \epsilon_t \sim NID(0, \sigma_\epsilon^2), \qquad t = 1, ..., n \\
\mu_{t+1} &= \beta_t + \mu_t + \eta_t, \quad \eta_t \sim NID(0, \sigma_\eta^2), \qquad t = 1, ..., n \\
\beta_{t+1} &= \beta_t + \xi_t, \quad \xi_t \sim NID(0, \sigma_\xi^2), \qquad t = 1, ..., n
\end{aligned}
$$

where y_t is the final generated data used and μ_t represents the trend factor with γ_t representing the seasonal factor. *NID* stands for *"normally and independently distributed"* and the seasonal factor γ_t is constructed with a sine function as follows:

$$\gamma_t = s_t \times \sin\left(\frac{2\pi t}{s}\right), \tag{18}$$

with s_t being the standard deviation of $\mu_1, ..., \mu_{t-1}$.

Datasets by state space are created with the following function, found in DATA.R:

```
R> data <- State_space_data(len = datalen, wp = datalen,
  sd_epsilon = epsilon, sd_eta = eta, sd_Xi = Xi, seasonality = TRUE)
```

where LEN is the length of data, WP is the starting point of output data, SD_EPSILON, SD_ETA, and SD_XI stand for the standard deviations of ϵ, η, and ξ, respectively, and SEASONALITY is a switch for whether seasonal factors will be included in the state space data. We set the parameters for the state space datasets as $n = 10000$, $\sigma_\eta = 0.1$, $\sigma_\xi = 0.02$, $\alpha = 0.01$, and $\lambda = 0.02$. We also varied σ_ϵ according to various noise conditions. Figure 12 shows a few examples with their respective σ_ϵ values.

We have made available all datasets used in this study through our GitHub repository. However, please note that the Yahoo! benchmark datasets are excluded in the repository due to licensing restrictions. To access the excluded datasets, we refer the reader to the following [17] Fig. 13.

Acknowledgements This work was supported by the National Research Foundation of Korea (RS-2025-23524280). This article is licensed under the Creative Commons Attribution License (CCAL), allowing others to download, reuse, reprint, modify, distribute, and copy the article, provided proper attribution is given to the original authors and source. The code accompanying this article is licensed under the GNU General Public License (GPL) version 2 or version 3 or a GPL-compatible license.

Author contributions YK wrote the manuscript, ran the experiments. SK ran the experiments, wrote the code. KL wrote the manuscript, supervised and funded the research.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

References

1. Adibi, M., Shahrabi, J.: Online anomaly detection based on support vector clustering. International Journal of Computational Intelligence Systems **8**, 735–746 (2015)
2. Adikaram, K.K.L.B., Hussein, M.A., Effenberger, M., Becker, T.: Data Transformation Technique to Improve the Outlier Detection Power of Grubbs' Test for Data Expected to Follow Linear Relation. J. Appl. Math. (none), 1–9 (2015)
3. Charu, C.: Aggarwal, Outlier Analysis, 2nd edn Springer Publishing Company, Incorporated (2016)
4. Diederik, P., Kingma, M., Welling.: Auto-encoding variational bayes, (2022)
5. Grubbs, F.E.: Procedures for detecting outlying observations in samples. Technometrics **11**(1), 1–21 (1969)
6. Guha, S., Mishra, N., Roy, G., Schrijvers, O.: Robust random cut forest based anomaly detection on streams. In Maria Florina Balcan and Kilian Q. Weinberger, editors, Proceedings of The 33rd International Conference on Machine Learning, volume 48 of Proceedings of Machine Learning Research, pages 2712–2721, New York, New York, USA, 20–22. PMLR (2016)
7. Hinton, G.E., Salakhutdinov, R.R.: Reducing the dimensionality of data with neural networks. Science **313**(5786), 504–507 (2006)
8. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Comput. **9**, 1735–80 (1997)
9. Holt, C.C.: Forecasting seasonals and trends by exponentially weighted moving averages. International Journal of Forecasting **20**(1), 5–10 (2004)
10. Kim, S.: Github repository for osesd written in GitHub Repository for osESD written in R (2024)
11. Kim, S.: Pypi link for osesd written in python. Pypi link for osESD written in Python (2024)

12. Kloft, M., Laskov, P.: Online anomaly detection under adversarial impact. In Yee Whye Teh and Mike Titterington. (eds.) Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, volume 9 of Proceedings of Machine Learning Research, pages 405–412, Chia Laguna Resort, Sardinia, Italy, 13–15. PMLR (2010)
13. Kramer, M.A.: Nonlinear principal component analysis using autoassociative neural networks. AIChE J. **37**, 233–243 (1991)
14. Leveni, F., Weigert Cassales, G., Pfahringer, B., Bifet, A., Boracchi, G: Online isolation forest. In Proceedings of the 41st International Conference on Machine Learning, pages 27288–27298. PMLR (2024)
15. Liu, FT., Ting, KM., Zhou, ZH.: Isolation forest. In 2008 Eighth IEEE International Conference on Data Mining, pages 413–422 (2008)
16. Rosner, B.: Percentage points for a generalized esd many-outlier procedure. Technometrics **25**, 165–172 (1983)
17. S5 - a labeled anomaly detection dataset, version 1.0(16m). Nikolay Laptev Saeed Amizadeh, (2015)
18. Dunning, T., Friedman, E.: Streaming Architecture. O'Reilly Media (2017)
19. Vallis, O., Hochenbaum, J., Kejariwal, A.: A novel technique for long-term anomaly detection in the cloud. In Proceedings of the 6th USENIX Conference on Hot Topics in Cloud Computing, HotCloud'14, page 15, USA, 2014. USENIX Association
20. Weixing, S., Liu, F., Zhao, J., He, M., Chen, H.: An on-line detection method for outliers of dynamic unstable measurement data. Clust. Comput. **22**, 07 (2019)
21. Wette, S., Heinrichs, F.: Oml-ad: Online machine learning for anomaly detection in time series data, https://arXiv:2409.09742 arXiv preprint (2024).
22. Winters, P.R.: Forecasting sales by exponentially weighted moving averages. Management Science **6**(3), 324–342 (1960)
23. Xu, L., Zhang, P., Xu, J., Wu, S., Han, G., Xu, D.: Conflict analysis of multi-source sst distribution. In: Zhang, W., Chen, Z., Douglas, C.C., Tong, W. (eds.) High Performance Computing and Applications, pp. 479–484. Springer Berlin Heidelberg, Berlin, Heidelberg (2010)
24. Xu, Y., Ma, Y., Zhang, K., Yang, Z., Ting, KM.: Idk-s: Incremental distributional kernel for streaming anomaly detection. In Proceedings of the AAAI Conference on Artificial Intelligence (2025)
25. Yahoo: Historical Stock Prices for APPL. Yahoo Finance, New York , USA (2023)
26. Yahoo: Historical Stock Prices for TSLA. Yahoo Finance, New York City, USA (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.